USING MACHINE LEARNING FOR RASPBERRY PI NETWORK INTRUSION DETECTION

A Thesis

Presented to the

Faculty of

California State Polytechnic University, Pomona

In Partial Fulfillment

Of the Requirements for the Degree

Master of Science

In

Computer Science

By

Angel Jimenez

2021

SIGNATURE PAGE

THESIS: USING MACHINE LEARNING FOR RASPBERRY PI NETWORK INTRUSION DETECTION

AUTHOR: Angel Jimenez

DATE SUBMITED: Spring 2021

Department of Computer Science

Dr. Abdelfattah Amamra
Thesis Committee Chair
Computer Science

_____

Dr. Mohammad I Husain
Computer Science

_____

Dr. Gilbert Young
Computer Science

_____

ACKNOWLEDGMENTS

This thesis is dedicated to my family, friends, and professors who have supported me throughout my education. A special thank you to my wife who has helped me by cheering me on and supporting me throughout my journey.

ABSTRACT

With the rapid growth in the scale of networked computers and existing apps, it has become clear that the possible harm caused by launching cyber-attacks is a great risk that has only increased in recent years. For this reason, the Intrusion Detection System (IDS) is a critical security control which can be used to identify advanced and continuously evolving network attacks. For network protection, a Network Intrusion Detection System (NIDS) allows a user to identify and respond to malicious traffic. The biggest advantage of an intrusion detection system is the ability to generate alerts and forward them to the security operations center. Once identified, incident response teams are alerted to investigate a possible attack or network breach. However, traditional signature-based IDS lack the ability to identify evolving network attacks. In addition, security tools such as firewalls and anti-virus applications usually do not have access to encrypted keys and rely on signature-based detection methods, making it difficult or impossible to specifically detect maliciously encrypted files. This study is conducted in order to capture the network traffic using the Zeek open-source platform that ensures effective capturing of traffic by generating logs. The network logs track key activities of the network which can be used to troubleshoot, perform network forensics, and display network-related information. The captured logs are then processed and labeled to classify anomalous network activity based on traffic of IoT devices deployed in a Smart Home environment. The machine learning library Sci kit learn is then used to train models for the experiments.

TABLE OF CONTENTS

SIGNATURE PAGE ii

ACKNOWLEDGMENTS iii

ABSTRACT iv

LIST OF FIGURES vii

1. INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Thesis Organization 2

2. LITERATURE SURVEY 4

2.1 Anomaly Based Network Intrusion Detection System 4

2.2 Machine Learning 8

2.3 Machine Learning for Network Intrusion Detection 16

3. RESEARCH GOAL 22

3.1 Research Objective 22

3.2 Proposed Work 22

3.3 Hypothesis 23

3.4 Experiments 24

4. METHODOLOGY 26

4.1 Smart Home Setup 26

4.2 Intrusion Detection Process 28

4.3 Configuring Raspberry Pi for Intrusion Detection 30

5. RESULTS 33

5.1 Supervised Learning.....33

5.2 Unsupervised Learning.....49

6. Conclusion and Future Work 52

REFERENCES 53

LIST OF FIGURES

Figure 1. Possible outcomes based on detection and event nature 6

Figure 2. Anomaly detection methods for analyzing network features.....7

Figure 3. Various behavior models used to analyze a system.....8

Figure 4. A diagram model of an artificial neuron 10

Figure 5. A mathematical formula of an artificial neuron.....11

Figure 6. A simple threshold activation function for an artificial neuron.....11

Figure 7. A mathematical formula to calculate the weights of a firing neuron 12

Figure 8. A diagram of a multi-layer perceptron neural network 13

Figure 9. A diagram of an autoencoder neural network.....15

Figure 10. A summary of the various intrusion detection datasets reviewed 18

Figure 11. Formula for defining a TCP packet in a dataset 18

Figure 12. 10 of the best features in the collected data using SelectKBest class 19

Figure 13. 10 of the best features in the collected data using Extra Tree Classifier ..20

Figure 14. A list of IoT devices used for this work 27

Figure 15. A diagram describing the network architecture and IDS 28

Figure 16. A summary of the process used by the IDS. 30

Figure 17. Confusion matrix displaying results of trained autoencoder 50

Figure 18. Formula for the mean squared error (MSE).....50

Figure 19. Analysis of network traffic after calculating the mean squared error 51

1. INTRODUCTION

1.1 Background

The number of IoT devices has increased and changed the way we interact with the internet in our day-to-day life. In addition, innovations continue to enable the widespread adoption of internet connected devices in different sectors such as consumer products, manufacturing, transportation, healthcare, and critical infrastructure. The convenience these connected devices provides are an obvious benefit for society, but the risk it poses can have incredible negative impacts to individuals and organization. If devices are not adequately protected, this can range from financial loss, and in extreme cases to the loss of human life.

In order to protect these devices, Network Intrusion Detection systems for commercial use have been an area of focus for researchers and substantial advancements have been made in recent years. but there has been a lack of focus for home-based networks. More so, a rapid transition to a work-from-home model combined with the growing use of IoT devices has increased the attack surface and leaves companies vulnerable.

For 2020 Nozomi networks reported an increase of threats to IoT networks which is attributed to an increased use of IoT devices, the COVID-19 pandemic, and increased sophistication of cybercriminals. IoT malware known as botnets will continue to pose a threat for organizations going forward. At its peak Mirai botnet had infected 600,000 IoT devices and new botnets are continually developed. 2020 saw new botnets including Dark Nexus,Mukashi, LeetHozer, Hoaxcalls and Mozi.m were some significant malware identified. In addition, existing solutions for securing an IoT environment are too expensive and do not offer the ability to detect zero-day attacks since they primarily leverage tradition signature-based detection.

1.2 Problem Statement

This research focuses on closing this gap between IoT devices and home-based intrusion detection by proposing the use of machine learning techniques and inexpensive hardware to detect malicious network activity in a Smart Home environment.

The use of machine learning allows us to solve this problem in a novel way. By implementing a special type of artificial neural network called an autoencoder, two main challenges are addressed. The first challenge being the lack of training data and the second is the ability to easily detect anomalies. An machine learning algorithms allows a model to effectively learn the patterns in the traffic generated in a day-to-day environment. This is important since every IoT Smart Home will vary based on factors including the number of devices, and the usage patterns. However, once trained, a model can be deployed in-line with a Raspberry Pi and analyze network traffic in real-time.

1.3 Thesis Organization

This thesis is organized by chapters that outline the implementation of machine learning techniques used to detect and classify malicious network traffic in a Smart Home environment. The Literature Survey chapter is a summary of work recently performed by various researchers and provides preliminary knowledge in the fields of network intrusion detection and machine learning. In Research Goal, the problem description is provided along with the experiments and expected outcomes. The Methodology chapter describes in detail how the experiments and expected outcomes are achieved. This includes an overview of how the lab environment was implemented, how the machine learning models were created and tested, and how the experiments were carried out for various IoT devices. The Results chapter presents the outcome and validation of the proposed solution. Finally, the Conclusion

and Future Work section summarizes the contributions of this thesis and provides a direction for future enhancements.

2. LITERATURE SURVEY

2.1 Anomaly Based Network Intrusion Detection System

2.1.1 Intrusion Detection System

An Intrusion Detection System (IDS) is a security tool used to detect common cybersecurity attacks. If the system is responding to events, it can be considered an Intrusion Prevention System (IPS). In addition, an IDS is typically classified as network-based or host-based. An Anomaly-based Intrusion Detection System (A-IDS) is a security mechanism used to detect abnormal activity for a host machine or a network by using heuristic and statistical techniques. Unlike a signature-based system, anomaly-based detection techniques offer the added benefit of detecting intrusions which were previously unknown. In the previous work of J. M. Estevez-Tapiador et al., three categories of detection techniques are described including statistical -based, knowledge-based, and machine-learning based [3]. This work primarily focuses on the neural network branch of the machine learning to implement an anomaly-based network IDS (A-NIDS).

2.1.2 Architecture for Anomaly Detection System

An anomaly detection system is comprised of two modules: modeling subsystem and detection subsystem. Formally, system γ can be defined as $\gamma = (M,D)$, where M represents the model of normal behavior in a system and D is the detection process [3]. Model M is obtained through a training phase in which a sensor is deployed to collect network traffic and trained as normal. Once M has been obtained, new packets are collected and compared using M to generate a similarity measure. This measure is used identify anomalous events in relation to model M. The methodology section of this work

describes the methods used to collect, train, and analyze network traffic in a Smart Home environment.

2.1.3 Classifying Anomaly Detection Methods in Networks

In order to successfully perform anomaly detection, all possible abnormal activities must be detected while reducing the number of false positives. This means we must consider all possible outcomes when classifying network traffic, which are false positives and negatives as well as true positives and negatives. False positive are normal events which are classified as abnormal events in the system and false negatives represent abnormal activities which are classified as normal events in the system. In order to reduce these to an acceptable level, it is important to have the most accurate possible model of normal behavior in a system. One way to achieve this is to systematically collect a large amount of network traffic representing normal behavior for the system which is described in the methodology section. True positives and negatives are indicators of a correctly functioning system. Fig. 1 is a summary of all possible outcomes used to detect and classify a suspected attack [3]. The following sub-sections describe analyzing network features and modeling behavior to classify network traffic as proposed by J. M. Estevez-Tapiador, et al. to describe the taxonomy for classifying anomaly detection methods in networks.

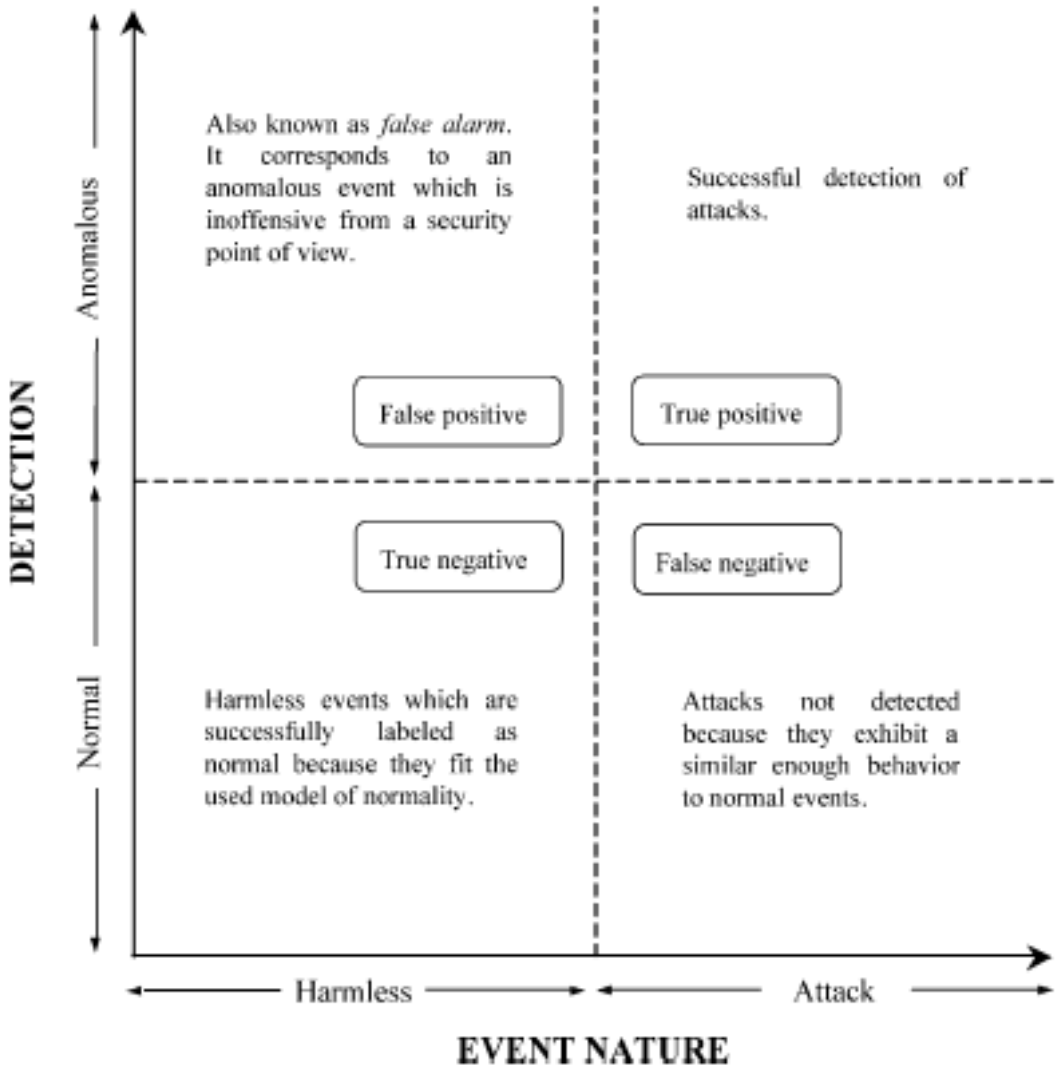


Figure 1. Possible outcomes based on detection and event nature to classify suspected attacks in a network using an intrusion detection system [3].

2.1.4 Network Features Analyzed

In order to successfully classify network activity as 'normal', a collection of network features must be defined. These features are then used to model the 'normal' behavior in the system. Since the work presented in this paper is tailored for a Smart Home environment, ad-hoc networks and how they connect are out of scope for this work. The focus is on a local area network (LAN) in which IoT devices are

connected and traffic is monitored with a Raspberry Pi running Zeek, a network security monitoring tool. Formerly known as Bro, Zeek is an open-source platform capable of providing transaction logs and customizable output for analysis of network traffic [36]. This makes Zeek an ideal tool for to perform flow analysis on features generated including number of bytes sent or received per connection, the number of packets processed, and the number of connections initiated. In addition, it can generate metrics for further protocol analysis at both the application layer and transport layer. Some of the various protocols easily analyzed include TCP, UDP, ICMP, HTTP, FTP, DNS, and SSH.

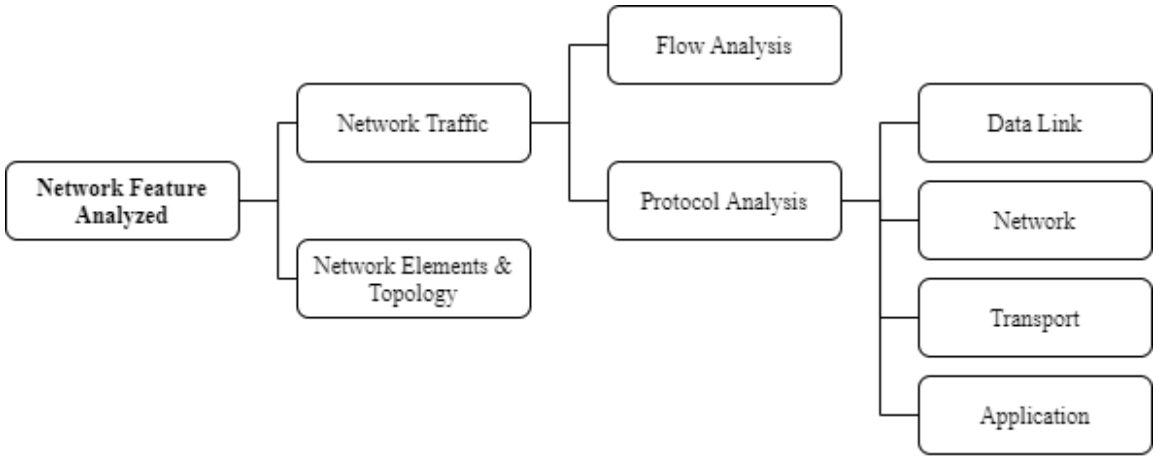


Figure 2. Anomaly detection methods for analyzing network features [3].

2.1.5 Behavior Model

The behavior model is another method which can be used to study and detect anomalies in a system. This model proposes using the normal activity of a system by observing it is in a secure state while looking for abnormal behavior. In order to achieve this, a model must be constructed independent of whether the feature that is modeled is a network protocol, traffic flows, or any other relevant characteristic. This can be done by using machine learning techniques to collect data and identify a

normal representation of the system which can then be used to identify new events that were not previously observed. By automatically identifying a "pattern of correctness", security professionals and experts are spared the task developing and specifications which can be difficult and time consuming [3].

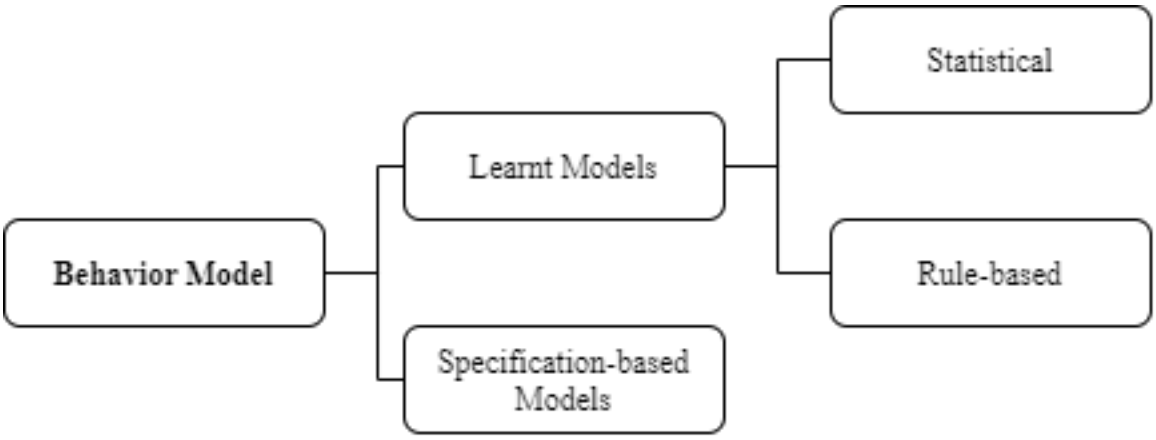


Figure 3. Various behavior models used to analyze a system [3].

2.2 Machine Learning

2.2.1 Supervised vs Unsupervised

Machine learning algorithms can be broken down into two different categories based on how they accomplish their learning. Murphy describes supervised learning as the most used in real world applications and can be used for classification or regression problems [22]. The work presented in this thesis focuses on the classification of malicious network activity, a classification problem, which requires the collection of data used as input into an algorithm, such as Random Forest or SVM described below. In Unsupervised learning, no input data is provided and instead an output is formulized by discovering interesting structures in the underlying data [22]. In this work, an autoencoder algorithm is used to explore the network traffic and identify patterns for intrusion detection.

2.2.2 Random Forest

A Random Forest algorithm is a supervised learning technique composed of a defined number of decision trees used to predict an outcome. It uses the average prediction of the trained decision trees to improve the accuracy and is commonly known to reduce overfitting [24].

2.2.3 Support Vector Machine

Another supervised learning technique is the Support Vector Machine. At the most basic level, a Support Vector Machine algorithm is trained by mapping inputs to points in space and maximizing the space between categories of data. Unlike a random forest, SVM can also be used as an unsupervised learning technique when labels are not provided [22]. Another additional benefit includes their effectiveness with high dimensional data such as network traffic [24].

2.2.4 Neural Networks

A neural network is a collection of mathematically modeled neurons consisting of inputs, weights, and an activation function. An ANN is based on a collection of connected units or nodes called artificial neurons, which loosely model the neurons in a biological brain. Each connection, like the synapses in a biological brain, can transmit a signal to other neurons. An artificial neuron that receives a signal then processes it and can signal neurons connected to it. The "signal" at a connection is a real number, and the output of each neuron is computed by some non-linear function of the sum of its inputs. The connections are called edges. Neurons and edges typically have a weight that adjusts as learning proceeds. The weight increases or decreases the strength of the signal at a connection. Neurons may have a threshold such that a signal is sent only if the aggregate

signal crosses that threshold. Typically, neurons are aggregated into layers. Different layers may perform different transformations on their inputs. Signals travel from the first layer (the input layer) to the last layer (the output layer), possibly after traversing the layers multiple times.

2.2.5 Neurons

In 1943, McCulloch and Pitts proposed the model in Figure 4:

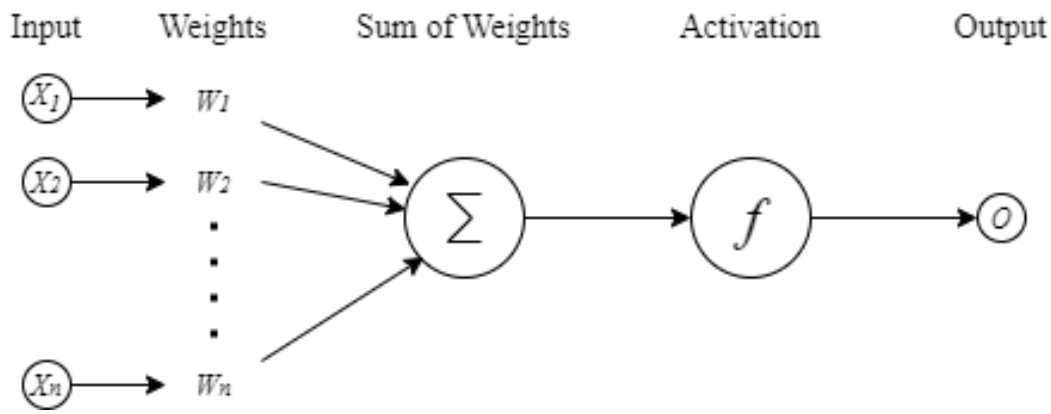


Figure 4. A diagram model of an artificial neuron as proposed by McCulloch and Pitts in 1943 [18].

Simplifying and modeling the function of a biological neuron and combining them into a neural network allows us to solve problems using machine learning. The following are important features to the model proposed by McCulloch and Pitts:

1) A set of weighted inputs W_i that correspond to the synapses
2) An adder that sums the input signals (equivalent to the membrane of the cell that collects electrical charge)
3) An activation function (initially a threshold function) that decides whether the neuron fires ('spikes') for the current inputs.

In Figure 4, the inputs X_i are multiplied by the weights W_i, and result is summed as the internal state of the neuron. When all these signals arrive at our neuron, it adds

them up to see if there is enough strength to make it fire. This can be represented with the following formula:

$$h = \sum_{i=1}^{m} w_i x_i,$$

Figure 5. A mathematical formula of an artificial neuron [17].

This sum is then used as input into an activation function. In this example the activation function is a simple threshold, if the sum is greater than the threshold, Θ then the neuron fires. If the threshold is not met, then the neuron would not fire, and the output would be 0. This can be written as:

$$o = g(h) = \begin{cases} 1 & \text{if } h > \theta \\ 0 & \text{if } h \leq \theta. \end{cases}$$

Figure 6. A simple threshold activation function for an artificial neuron [17].

A single neuron does not do anything more than fire according to its activation function. This does not make it very practical or useful on its own. However, a neuron is foundational to solving machine learning problems, including intrusion detection. By grouping and structuring a collection of neurons, we can train a computer program to learn and adjust its weights accordingly to produce a desired result. In the next section, we will see an example of a common neural network used.

2.2.6 Learning

Learning occurs by changing connection weights after all neurons have fired and calculating the error. This is based on the amount of error in the output compared to the expected result. Formally, the weights are updated according to the following formula as

described by:

$$w_{ij} \leftarrow w_{ij} - \eta(y_j - t_j) \cdot x_i.$$

Figure 7. A mathematical formula to calculate the weights of a firing neuron [17].

During learning, every weight connected to a neuron is updated according to the formula in Figure 7, where W_{ij} represents the weight of a neuron to be updated. The weights are then adjusted by taking the difference between the weight and the error function, or the difference between the output Y and the target. In addition, to account for negative inputs, we will multiply the error with the input before taking the difference. Finally, n represents the learning rate which determines how fast the network should learn. By repeating this process with every training example, the neural network will hopefully improve over time and the learning is complete once the performance stops improving.

2.2.7 The Multilayer Perceptron

The multilayer perceptron, is a neural network composed of sets of input, hidden, and output neurons using weighted connections. Figure 8 is a diagram showing the architecture of a multi-layer perceptron neural network:

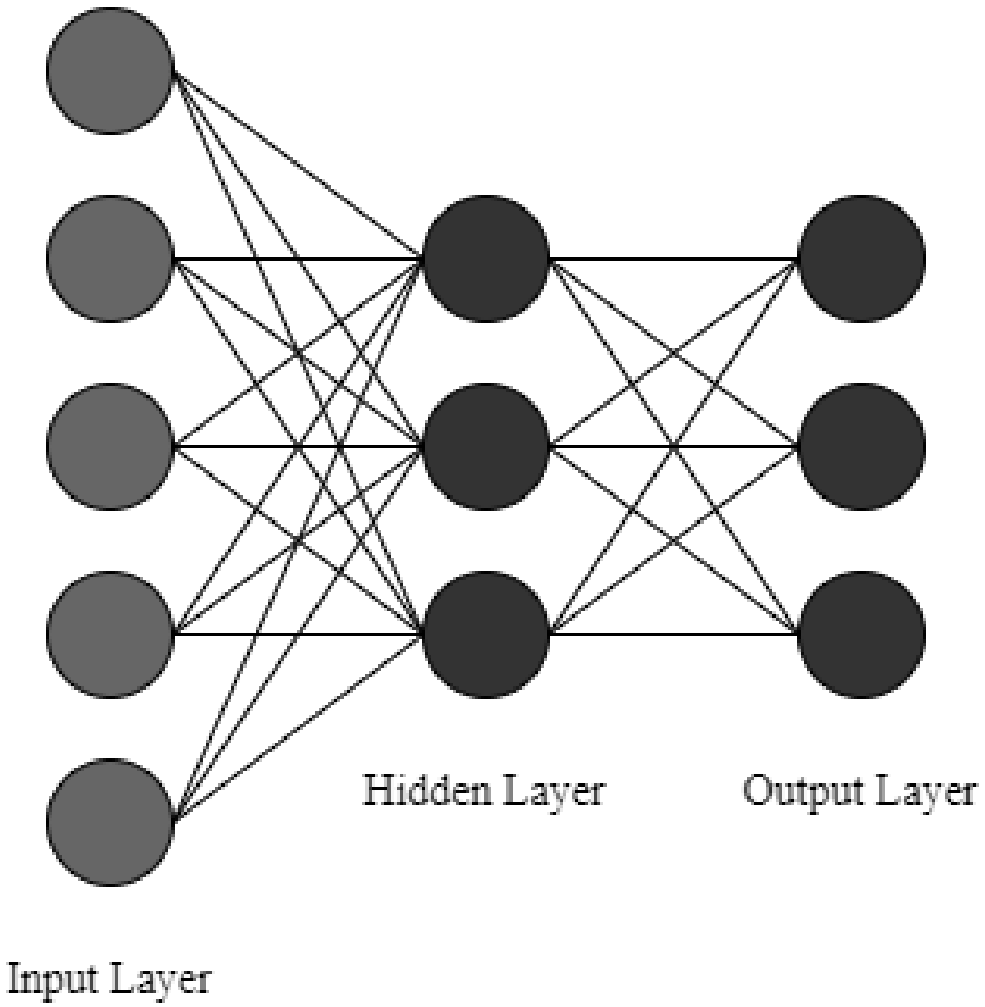


Figure 8. A diagram of a multi-layer perceptron neural network [17].

In this example, this perceptron has 5 inputs values (light circles), or a vector of 5 dimensions, a hidden layer consisting of 3 neurons (dark circles), and an output layer made up of 3 neurons. As we saw earlier, each neuron will either fire or not based on its activation function after using its own weights and multiplying the inputs before summing its result. It is important to note, this happens independently for each neuron and only the input is shared between each neuron. For this network to learn, it must be trained so that the weights adapt to generate the correct answers. This can be broken down into two parts:

1) Working out the outputs for the given inputs and the weights.

2) Updating the weights according to the error.

Part one requires simply setting the values of each input and performing formulas in Figure's 5 and 6 for each neuron in the network. The result is a vector of 1's and 0's which is then used to calculate the error by comparing the result to the expected values. This process is described in the following section.

2.2.8 Autoencoder Neural Network

Goodfellow et al. describe an autoencoder as a type of neural network trained using unsupervised learning. Unsupervised learning is a machine learning technique in which data is used to identify patterns without providing expected results, or labels. It is primarily used for dimensionality reduction and feature extraction in complex datasets. More specifically, an autoencoder learns by copying or predicting its input to its output and using a similarity measure to calculate the error as it learns. It accomplishes this by minimizing the reconstruction error after the hidden layer has captured the most relevant features of a dataset. Unlike a multilayer perceptron, an autoencoder must have hidden layers which are narrower than the input and output in order to avoid memorizing or mapping its input without learning. This allows for the autoencoder to extract the most important features within the dataset and use these representations to generalize well on new data [6].

2.2.9 How an Autoencoder Works

Unlike a multilayer perceptron, an autoencoder must have hidden layers which are narrower than the input and output in order to avoid memorizing or mapping its input without learning [13]. This allows for the autoencoder to extract the most important

features within the dataset and use these representations to generalize well on new data. By creating a “bottleneck” in the neural network, an autoencoder compresses the features extracted which is then used to reconstruct the input. M. A. Kramer describes this process as Nonlinear Principal Component Analysis and proposes an architecture like Figure 9.

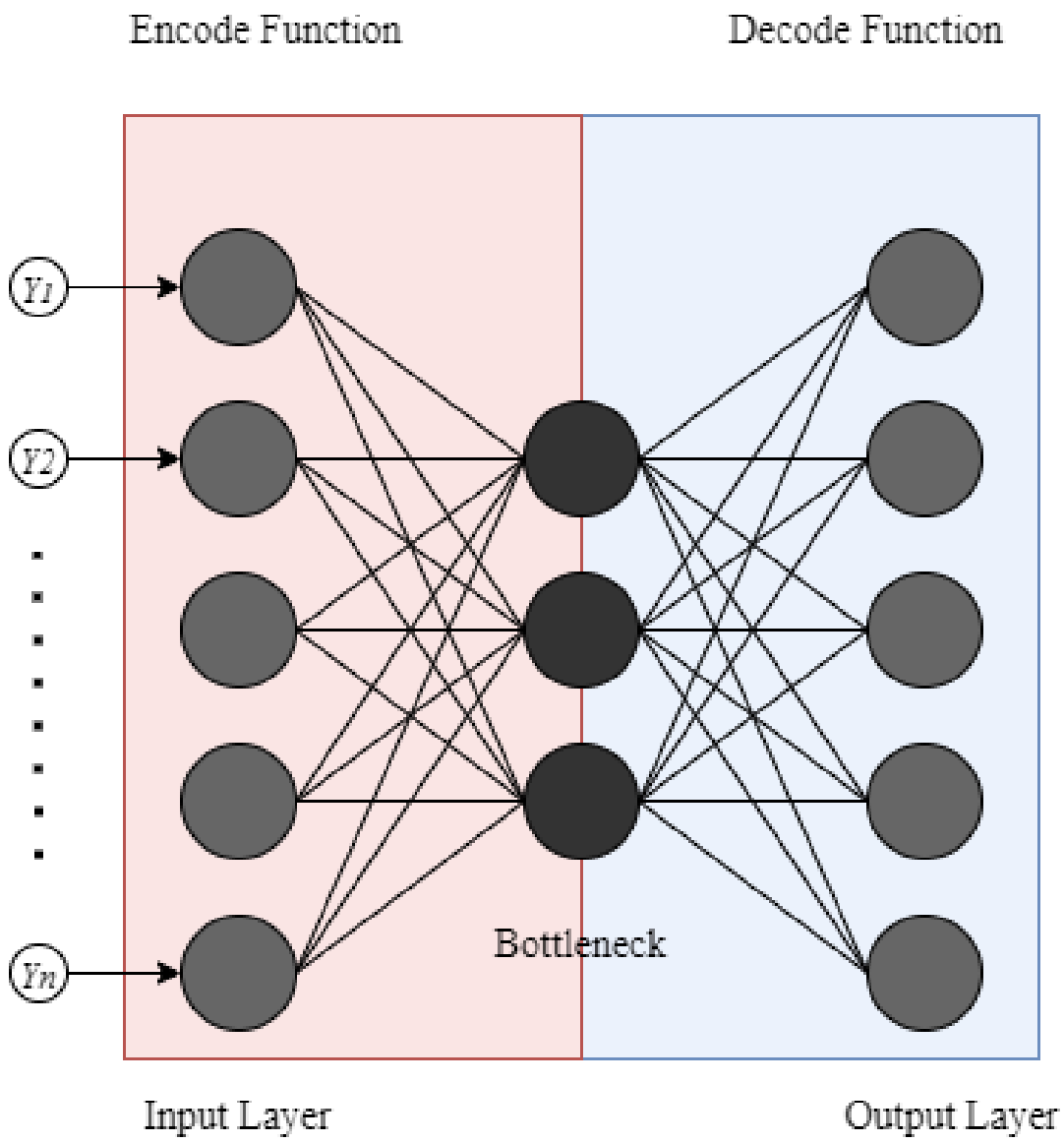


Figure 9. A diagram of an autoencoder neural network.

The key idea behind this architecture is to take the input layer and map it to a latent space using an encoding. When the data provided is well structured, the "bottle neck" layer is used to reduce the dimension of the data and will hold a reduced representation of the input. The decoding step then uses the latent space of lower dimensions and reconstructs it back to an output with the same dimensions of the original input.

2.3 Machine Learning for Network Intrusion Detection

In the domain of network intrusion detection, the ability to process large amounts of data in a complex and unique network environment presents several challenges. Recent work by Shone et al. outlines some of these challenges and highlights the need for novel techniques. Despite major advancements in the security industry, majority of existing intrusion detection systems rely on signature-based detection. By relying on these techniques, institutions face a higher risk of ineffectively and inaccurately detecting intrusions. By focusing research on techniques such as neural networks and autoencoders, institutions can begin to address the challenges which include the lack of widely available or outdated datasets, the inability to detect previously unknown attacks, and the need to tailor training to each unique environment.

2.3.1 Gathering Network Traffic Data

One of the common challenges faced by researchers in the domain of network intrusion detection is the availability of public datasets. Since every network is custom and configured for a specific environment, there is no single available dataset to accurately represent every network when training models. In addition, creating a dataset

and labeling network traces is a difficult and time-consuming task. This is required when training an algorithm using supervised learning techniques. Fortunately, we can address this challenge by using unsupervised techniques like autoencoders. This prevents the reliance on third party datasets and allows to create a true representation of the deployed network environment.

By collecting network traffic locally over a period, packets can be pre-processed and used for training only normal traffic. The approach presented in this project and described in the Methodology section focuses on capturing real-time network traffic in a smart home IoT environment and using this data to train an autoencoder based on normal day-to-day activity of network usage. However, during this research, various industry standard datasets were studied. These were used for testing, evaluation, and comparing techniques. Figure 10 shows a summary of the comparison between some widely used datasets currently available for intrusion detection. The process used to select features will be discussed further in the Methodology section.

Dataset	# of Attack Types	Dataset Size	# of Features
CIC 2017	14	51.1 GB	83
CTU-13	13	696.9 GB	32
IOT-23	10	21 GB	21
KDD '99	24	743 MB	41
UNSW-NB15	9	100 GB	49

Dataset	Total Traffic (Flows)	Attack Traffic	Normal Traffic
CIC 2017	2.8 m	.5 m	2.3 m
CTU-13	20.5 m	.44 m	20.1 m
IOT-23	325 m	294.5 m	30.8
KDD '99	5 m	3.9 m	1.1 m
UNSW-NB15	257,673	164,673	93,000

Figure 10. A summary of the various intrusion detection datasets reviewed [5][7][20][26][30].

2.3.2 Feature Selection

Features for our autoencoder can be gathered by capturing network traffic data by processing raw network packets using tools like TCP dump or Wireshark in PCAP format. Those data collected can then be preprocessed and connections can be represented as records in a file to be used for machine learning. The connection is simply a series of well-defined protocols of TCP packets, which start and end at a certain point. Any record of connections is a vector and specified as:

$$V = \{F_1, F_2, \dots, F_n, C\}$$

Figure 11. Formula for defining a TCP packet in a dataset [21].

where, F_i represents i^{th} feature, n is the length of the feature set, C is the class label [21].

In addition, two techniques were used to identify features which had the most

contribution to predicting whether traffic was anomalous, univariate selection and feature importance. Univariate selection uses statistic tests to select those features that have the strongest relationship with the output variable. The scikit-learn library provides the SelectKBest class that can be used with different statistical tests to select a specific number of features [24]. The example, Figure 12, below uses the chi-squared (chi²) statistical test for non-negative features to select 10 of the best features in the collected data:

	Specs	Score
9	resp_ip_bytes	5.470049e+08
8	resp_bytes_sum	2.501372e+08
5	orig_bytes_sum	1.730334e+08
2	id.resp_p	9.933368e+07
0	duration	9.791166e+07
6	orig_ip_bytes	8.602201e+07
1	id.orig_p	3.965157e+07
10	resp_pkts	1.427887e+07
4	no_of_flows	1.308134e+06
7	orig_pkts	1.193565e+06

Figure 12. 10 of the best features in the collected data using SelectKBest class [24].

Feature importance gives you a score for each feature of your data, the higher the score more important or relevant is the feature towards your output variable. Feature importance is an inbuilt class that comes with Tree Based Classifiers, this work used the Extra Tree Classifier for extracting the top 10 features from the dataset [24].

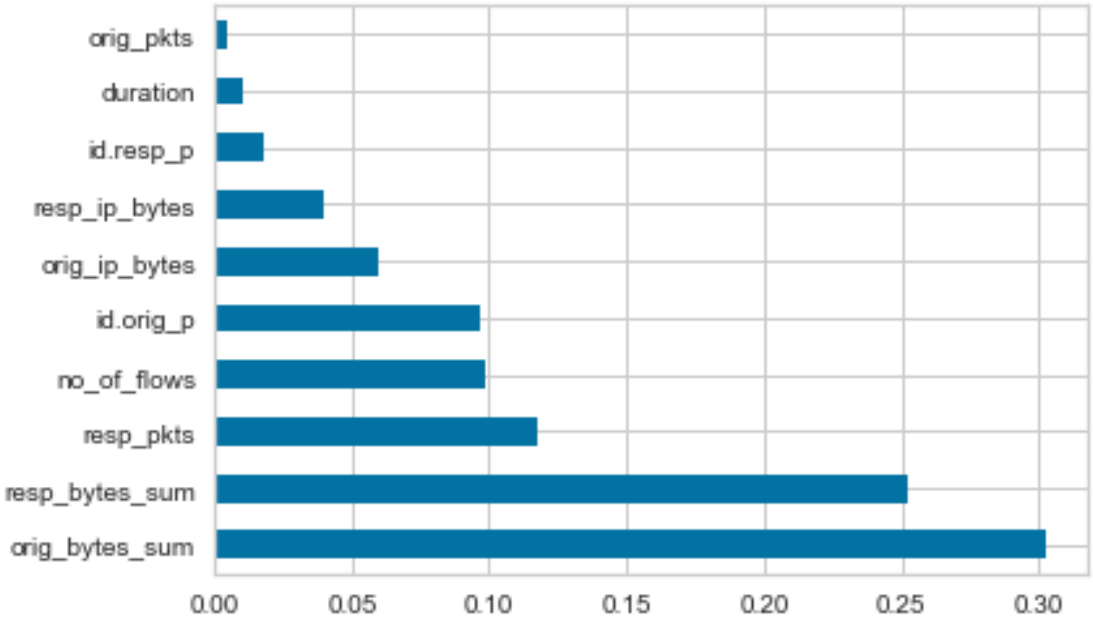


Figure 13. 10 of the best features in the collected data using Extra Tree Classifier [24].

2.3.3 Autoencoder for Baselining Normal Network Traffic

In the traditional signature-based approaches, NIDSs maintain a database of known attack signatures. When a signature matching an entry in the database is detected, the system generators an alarm to be reviewed by an administrator. It is important to note that signature-based NIDSs are unable to detect new attacks because a signature for new attacks will not be in the database.

Anomaly-based NIDSs use statistics and machine learning to create a baseline of system activity for behavior deemed normal. By using this baseline, network activity is compared to a predefined threshold and will generate an alarm for any activity which surpasses the threshold. This makes anomaly-based NIDSs capable of detecting new attacks. Establishing a baseline of normal network traffic can be done using an autoencoder. This technique requires training an autoencoder over a period and allows the autoencoder to learn what constitutes normal behavior in a network.

By doing so, the autoencoder can detect anomalies and generating an alert whenever behavior exceeds a defined threshold. In addition, this approach is can detecting new attacks. The use of anomaly-based detection is supported by previous work by Summerville et al. They argue that zero-day attacks can overburden databases in a resource constrained environment, making it a good option over signature-based detection approaches which are not well suited for an IoT Smart Home environment. In addition, IoT devices present a less complex network over networks used for general computing. This makes it possible to leverage machine learning techniques like autoencoders to create an accurate representation of normal behavior for detection anomalies [31].

3. RESEARCH GOAL

3.1 Research Objective

The objective of this work is to deploy a Smart Home IoT environment and train different types of machine learning algorithm to identify anomalies based on network activity. This includes capturing 'normal' network traffic over a specified period by deploying a network sensor to capture network packets as well as introducing malicious traffic. Once trained, the sensor will then be used to capture live network traffic and perform intrusion detection. The research goals are as follows:

1) Deploy a series of IoT devices segregated on a home network.
2) Capture network traffic over a period which captures the 'normal' network traffic activity.
3) Identify network features to be used for training data and label data collected.
4) Train machine learning algorithms and deploy in the Smart Home environment with live network traffic and test against various 'anomalous' scenarios.
5) Test and document the results for various IoT devices in the environment.

3.2 Proposed Work

In order to meet the goals presented in the preceding section, an existing home network is modified with inexpensive hardware to provide the capability of collecting network traffic. This includes the capability to monitor network traffic via "port mirroring". An inexpensive or used switch can provide this capability. The

experiments presented in this work used an 8-port Unifi Switch and Unifi Access Point. In addition, a Raspberry Pi 4 was deployed with Zeek installed to collect network traffic and Python was used to perform various machine learning tasks. These modifications and configurations are described in detail in the Methodology chapter.

With the correct infrastructure in place, various IoT devices were deployed and operated under both normal and abnormal conditions while network traffic was collected using Zeek to generate the logs with network connection information. Collecting this data served two purposes. The first purpose is as an initial training data set for the machine learning algorithms which consists of identifying key features to be used and training the machine learning algorithms with those features. The second purpose is to develop a data pipeline for collecting the live network data and preprocessing into a machine learning format by encoding and normalizing the features.

3.3 Hypothesis

The hypothesis is that using a Raspberry Pi 4 preconfigured with Zeek IDS and combining the log analysis with machine learning techniques is an effective method of identifying abnormal network traffic activity in a Smart Home environment for real-time alerting. By leveraging supervised and unsupervised machine learning algorithms, network traffic can be collected and classified as benign and malicious to effectively alert a user of potential malicious activity. In addition, supervised learning techniques will provide a reliable method for detection with high accuracy while unsupervised may be used to identify interesting patterns in

the network data.

3.4 Experiments

The primary goal of the experiments in this work is to demonstrate the ability for a solution to detect abnormal activity by analyzing logs generated in the Zeek environment using machine learning. In order to achieve this, two common IoT attacks are performed against several devices in the environment. The first is a Denial-of-Service attack which is used to disrupt the functionality of a device and potentially cause damage in critical systems. To perform this attack, the Hping3 tool was used to send TCP/IP packets to the victims using the following command:

*hping3 -c 1000 -d 120 -S -w 64 -p 0 --flood --rand-source *victim IP address**

The parameters provided to the tool can be broken down as follows:

1) The *-c* flag allows a user to define the number of packets sent, 1000 packets in this case.
2) The *-d* flag defines the size of each packet sent to a targeted device.
3) -S instructs the tool to send only SYN packets
4) -w sets the TCP window size to 64
5) -p allows a target port to be defined
6) –flood, sends packets as fast as possible and disregards incoming replies
7) –rand-source, generates random IP address to spoof the attacker machine

The second is a reconnaissance attack used to discover victims on a network and identify targets for lateral movement. To perform this scenario, the Nmap tool was used scan a target machine for the purpose of identifying information such as manufacturer,

open ports, services actively running, and identifying the operating system version. The following command was used to perform this experiment:

*nmap -sS -sU -T4 -A -v *victim IP address**

1) -sS instructs the tool to perform a TCP Syn scan which is commonly used for its unobstructive and stealthy nature.
2) -sU, is used to perform a scan on the UDP protocol
3) -T4, is used to improve efficiency and considered “aggressive.”
4) -A, enable OS and version detection.
5) -v is used to increase verbosity level for more detailed output.

4. METHODOLOGY

4.1 Smart Home Setup

4.1.1 IoT Devices

IoT devices used in this experiment range in manufacturers and functions in order to replicate a typical smart home environment. All devices were purchased for home use and the network traffic captured for analysis is generated from day-to-day usage. In addition, devices were added and removed throughout this work which allows the opportunity for additional research in terms of retraining techniques. For purposes of this experiment, IoT devices were purchased directly from certified retailers and does not consider the possibility of a supply chain compromise. The devices used were also updated regularly and configured according to manufacturer recommendations. An important note to make is that traffic generated by each device may vary significantly. For example, when performing a network traffic capture, the traffic generated by the Amazon Firestick TV heavily outweighed the typical usage of other devices. To account for this variation, individual models were trained for each device used in the experiments. A benefit of this approach is the ability to add and remove devices from a network without the need to retrain the entire network. A full list of devices used are listed in Figure 14.

Manufacturer	Model	Function
Google	Nest	Thermostat
Google	Nest Hello	Doorbell
Google	Home	Assistant
Roomba	675	Vacuum
Amazon	Firestick	TV
Geeni	Smart Bulb	Smart Bulb

Figure 14. A list of IoT devices used for this work.

4.1.2 Network Topology

As mentioned previously, minor network upgrades were required to effectively capture network traffic. This included the use of an Ubiquiti Unifi Switch with the capability to mirror network packets to a separate port. Another feature this switch provides is the ability to segregate the network into VLANs. This benefit adds a layer of security between the trusted portion of the network and the IoT network. In addition, an Ubiquiti Unifi Access Point was also installed which is used as a gateway for the various IoT devices installed in this environment. With the correct network infrastructure in place, a Raspberry Pi 4 with a gigabit ethernet port was connected to the mirroring port on the Ubiquiti switch using a standard ethernet cable. This model was specifically selected over previous models for its ability to process network traffic at gigabit speeds and avoid reaching the capacity provided by previous models such as version 2 or 3. A full network topology is show in Figure 15.

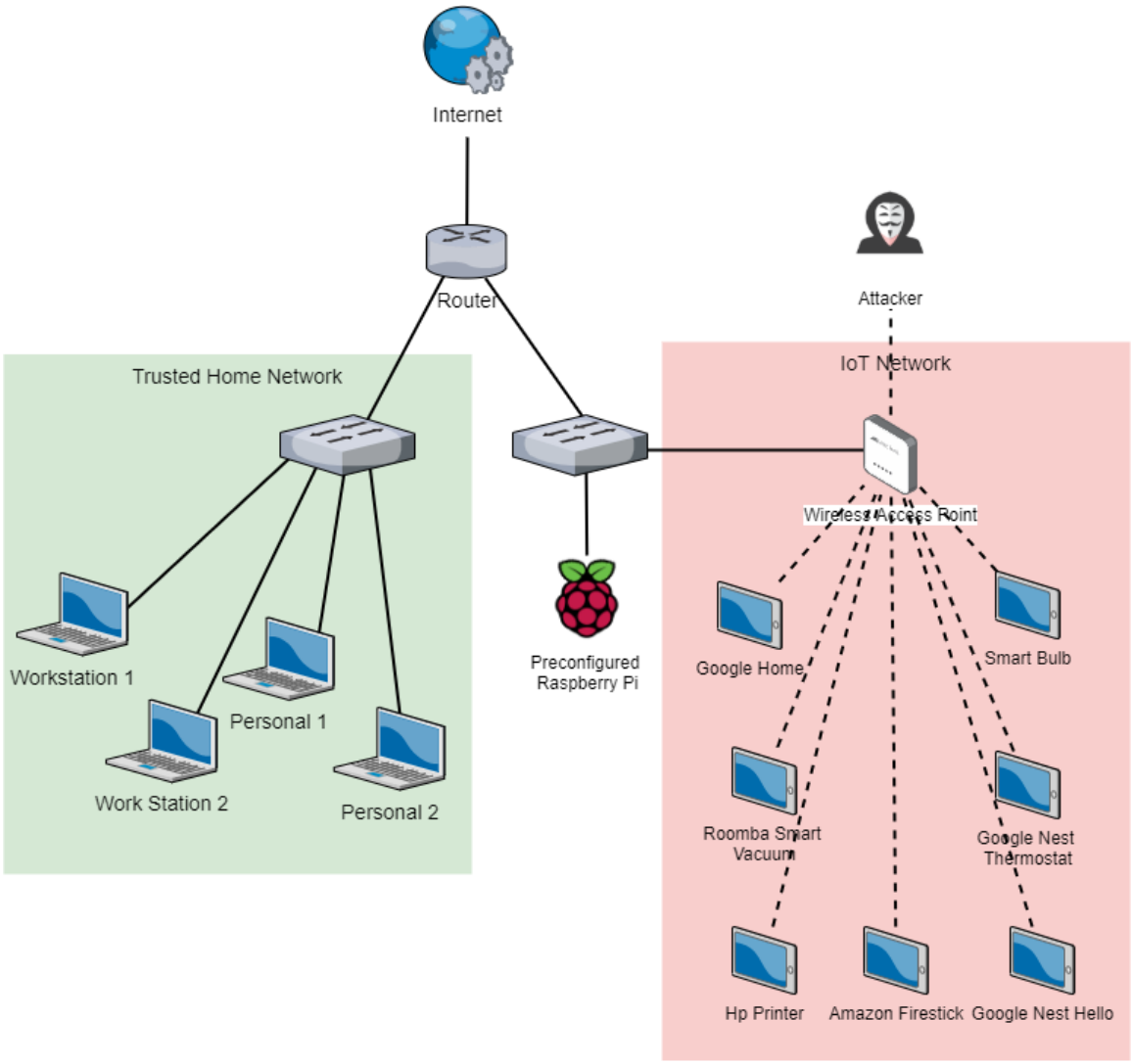


Figure 15. A diagram describing the network architecture and IDS.

4.2 Intrusion Detection Process

Figure 15 above also includes the intrusion detection system which consists of three primary functions which collect, analyze, and store real-time data generated by the IoT devices. Although data collection and analysis performed on the Raspberry Pi, an external hard drive was used to allow for additional storage space. The functions are described as follows:

1) Data collection – The data collection function of the intrusion detection system primarily relies on collecting network packet captures and processing these in desired intervals by using Zeek, an open-source network analysis tool. This allows the system to efficiently capture traffic and preprocess the network data into log files generated by Zeek. The primary benefit to using Zeek for this process is the ability to extract data from the log files used to record network activity for every connection made in the network. The ability to generate these logs makes Zeek an attractive tool for use with machine learning which is described in the Zeek section below [36].
2) Data Storage – The data storage step is simply used to store logs and serve as a queue to process logs generated during the previous step. These logs are read and managed by Zeek in a compressed format.
3) Data Analysis – The data analysis function is also two-fold. The first task performed is to convert the logs collected into Pandas Dataframes. Jupyter Notebook is a popular python library which includes Pandas which is used for data analysis and manipulation. Jupyter Notebook allows us the flexibility to manipulate the data and generate features using additional tools NumPy, Sci-Kit Learn, and MatplotLib [12]. Once the data is machine-learning ready, a model of the trained autoencoder is used to perform inference on the generated data. The model is trained using TensorFlow and runs locally on the Raspberry Pi during the analysis step [2]. Finally, once the inference is complete a message is displayed indicating whether any anomalies were detected. The entire intrusion detection process is shown in Figure 16 and the individual steps are further describe in the following section.

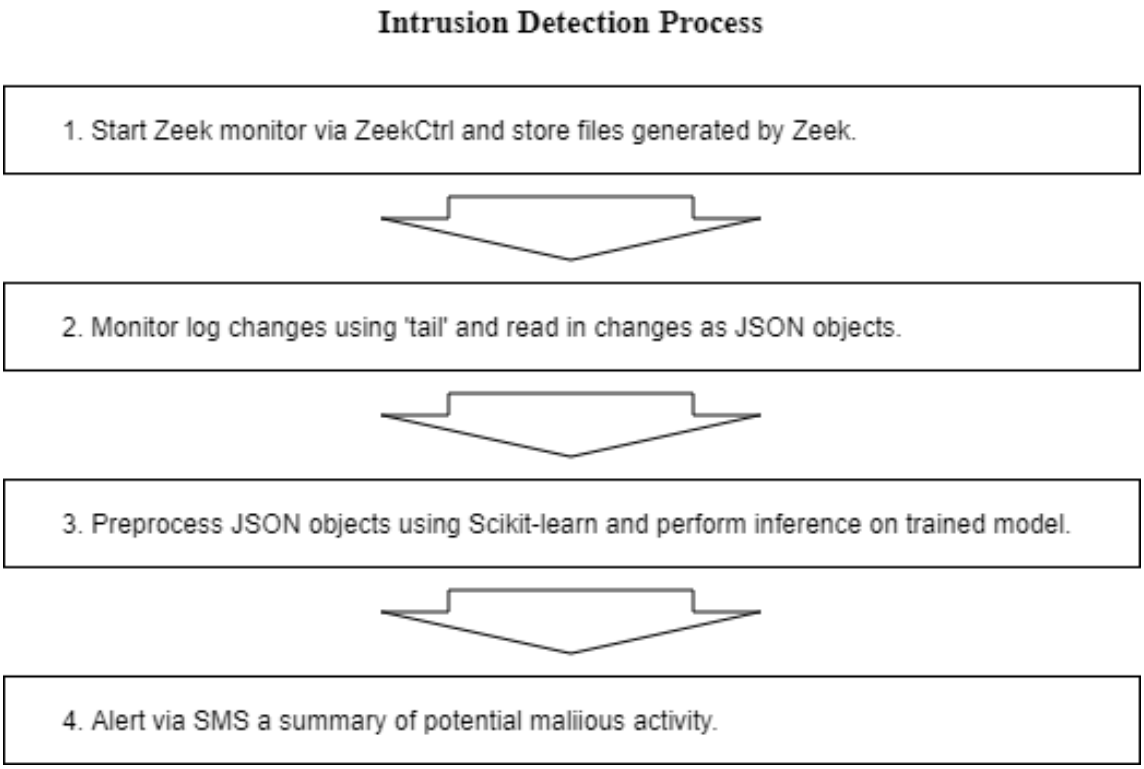


Figure 16. A summary of the process used by the network intrusion detection system.

4.3 Configuring Raspberry Pi for Intrusion Detection

The Raspberry Pi is a low-cost ARM computer system developed by the Raspberry Pi Foundation and first released in February 2012. Although primarily used to teach fundamentals of computer science in developing countries, its small footprint and extensive features make it a powerful platform for researchers in developing proof of concepts. The Raspberry Pi 4 Model B, or RPi4, used in this thesis features a 4GB RAM, 1.5GHz arm_v7 processor, USB3, WiFi, Bluetooth, and Gigabit ethernet connectivity.

In order to convert this device into a capable intrusion detection system, various open-source software, tools, and libraries were installed. These tools were used to monitor network traffic, parse and convert log files to a machine learning format,

perform inference on the live data feed, and finally send alerts to the user. These tools are described in detail in the following sections.

4.1.3 Generating Logs with Zeek

Zeek is an open-source platform for performing traffic analysis and security monitoring. It is a powerful protocol analyzer out of the box and is easily deployed to the Raspberry Pi. By default, Zeek is configured to monitor interface 'eth0' which is used by the deployed Raspberry Pi to monitor network traffic from the switch performing port mirroring. Once connected, logs are captured and aggregated in the 'logs' directory. This location is '/usr/local/zeek/logs/' for the solution presented in this thesis. In additional, the default output format was modified to output logs in JSON format and the 'conn.log' file was used as the primary data source for data collected and used to train the machine learning algorithms. Although various log files are generated, 'conn.log' provided the most consistent information for each device which resulted in promising results.

4.1.4 Monitoring Zeek Logs with Tail

One of the key tasks for the Raspberry Pi Intrusion Detection system is to actively monitor the generated Zeek logs in real-time. This is achieved via the 'tail' command which is executed through Python as Zeek is actively analyzing and generating logs. By using the 'tail' command, changes to the log an be read and Parsed as JSON objects as input into the data pipeline.

4.1.5 Machine Learning with Scikit-learn

Scikit-learn is a popular machine learning library available for the Python programming language that offers a wide variety of machine learning algorithms. A

key benefit of Scikit-learn is its compatibility with other libraries such as NumPy and Pandas which are used in this work throughout the data pipeline. In order to achieve the goals for this work, several packages were used to train, test and deploy the machine learning models used in these experiments. The primary purpose for using this library is to train and deploy machine learning models on the Raspberry Pi. Scikit learn is key to achieving this task by providing various packages to preprocess, evaluate metrics, variety of algorithms. In addition, it is easily installed on the Raspberry Pi and can be efficiently used for real-time data processing. This task can be broken down further into a series of steps, or data pipeline, which require collecting the training data, performing feature selection to extract key features, splitting the data, convert the data into a machine learning format, train and test machine learning models, and reviewing the performance metrics. This process is then repeated until the desired metrics are reached. Finally, a trained model is easily stored as a file and can be imported when inference is needed.

4.1.6 Alerting via Twilio

Twilio is an easy-to-use SDK and API which can be used for various multimedia alert services. Twilio was primarily used in this project to send SMS alerts to a user which provide a summary of the potential malicious activity on the network. Although not open-source, Twilio offers a low-cost service capable of handling alerts with a trail credit for the user. In addition, alerts were limited to an 8-hour time frequency to avoid constant alerting in when malicious traffic or false positives are detected. Although open-source options are available, the ease-of-use and low cost of Twilio makes it an ideal solution for this research.

5. RESULTS

5.1 Supervised Learning

The primary goal of a classification algorithm in the context of network intrusion detection is to achieve the highest level of accuracy with the lowest number of false positives. In addition, the True Positive Rate (TPR) (also referred to as Detection Rate) is an important metric for network intrusion detection as it indicates the number of malicious examples that are correctly identified. Several common metrics are used to evaluate the effectiveness of the deep learning approaches are [27]:

1) True Positives: Attack data that is correctly classified as an attack.
2) True Negatives: Normal data that is correctly classified as normal.
3) False Positives: Normal data that is incorrectly classified as an attack.
4) False Negatives: Attack data that is incorrectly classified as normal.

The performance of a machine learning algorithm can be depicted via a confusion matrix.

In the context of network intrusion detection, metrics can be defined as follows [27][34]:

- True Positive Rate (TPR) or Recall: $TPR = Recall = \frac{TP}{TP+FN}$.
- True Negative Rate (TNR): $TNR = \frac{TN}{TN+FP}$.
- False Positive Rate (FPR): $FPR = \frac{FP}{FP+FN}$.
- False Negative Rate (TNR): $FNR = \frac{FN}{FN+TP}$.
- Accuracy is the ratio of the number of total correct predictions made TP + TN to all predictions made TP + FP + FN + TN, or: $\frac{TP+TN}{TP+FP+FN+TN}$.
- Precision is the ratio of the total number of correctly predicted positive classes TP to the total number of positive predictions made TP + FP, or:

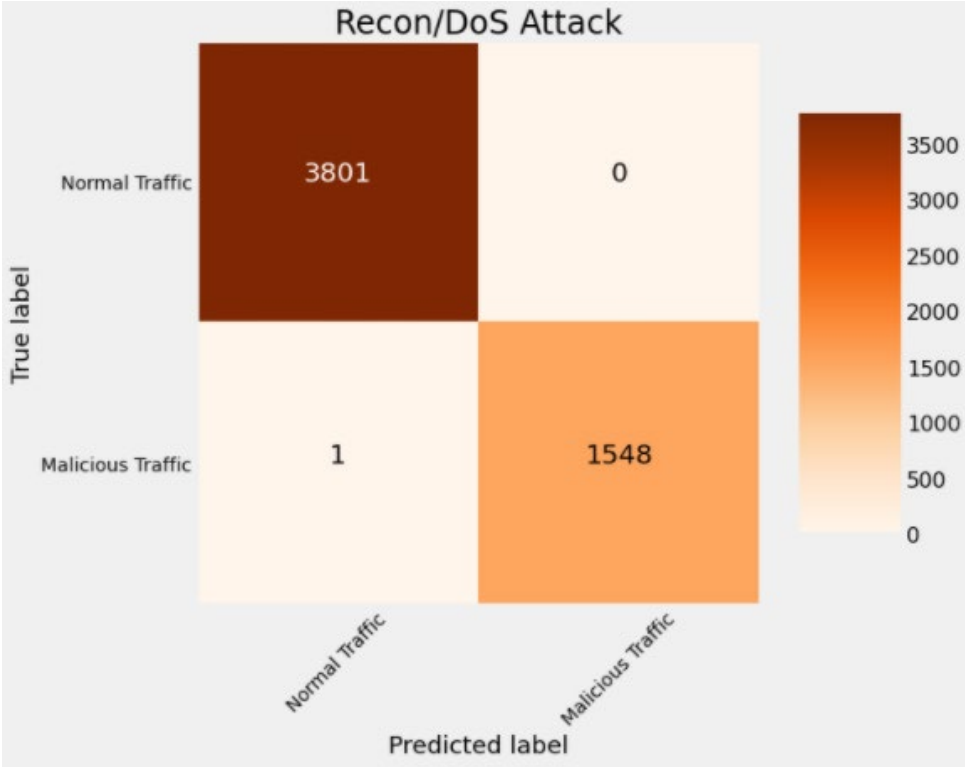
$\frac{TP}{TP + FP}$.

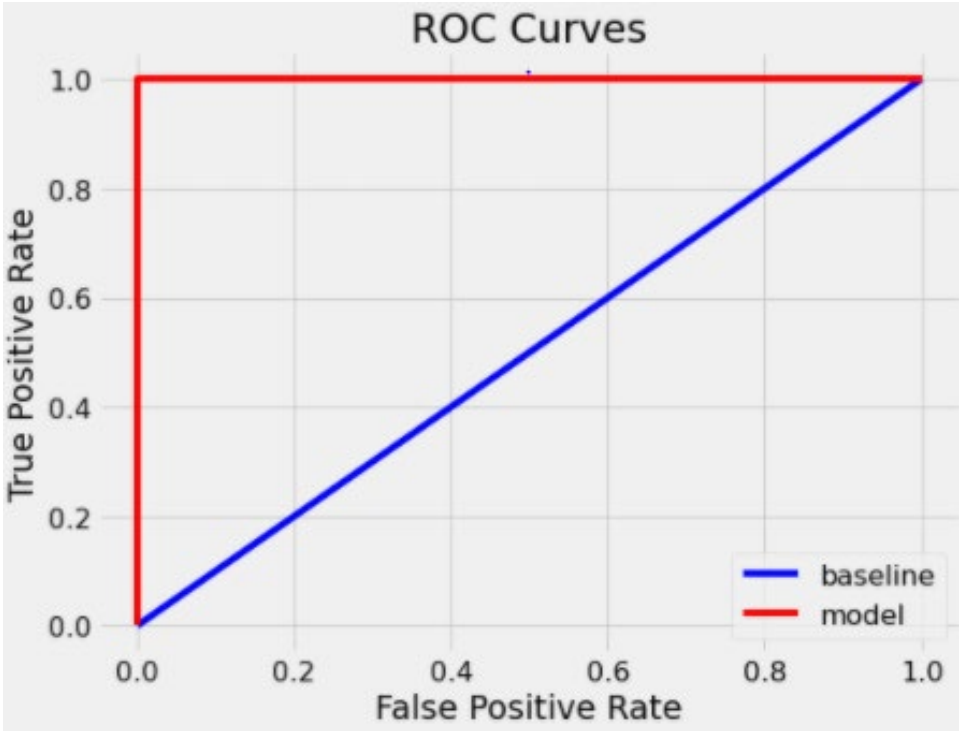
- The Receiver Operating Characteristics (ROC) curve is a plot of True Positive Rate (TPR) on the y-axis against False Positive Rate (FPR) on the x-axis.

The experiments in this research were carried out using the algorithms describe in the Literature Survey section; Random Forest, Support Vector Machine, Multi-Layer Perceptron. Each device was individually targeted and using the commands described in the experiments section of the Research Goal section.

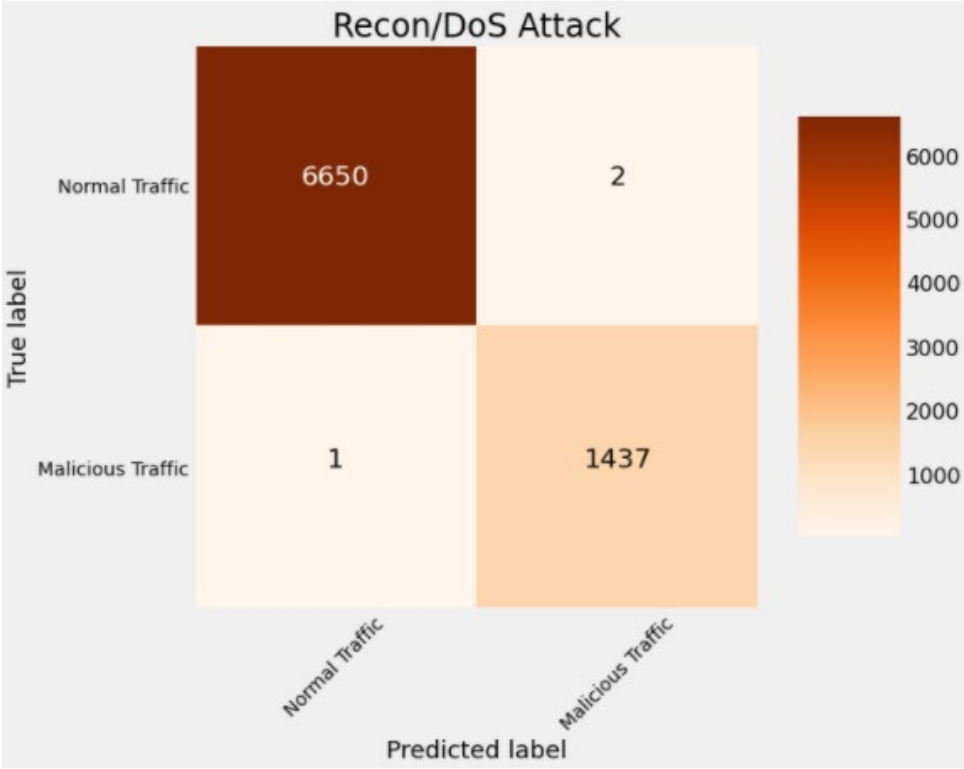
5.1.1 Random Forest Confusion Matrices and ROC Curves

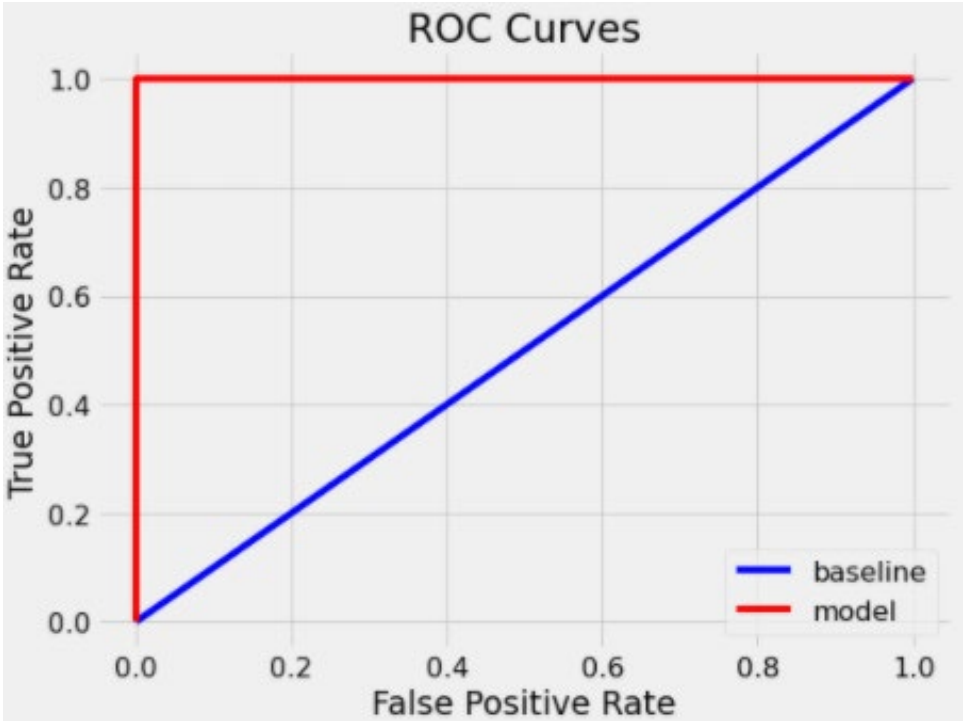
Smart Bulb Device:



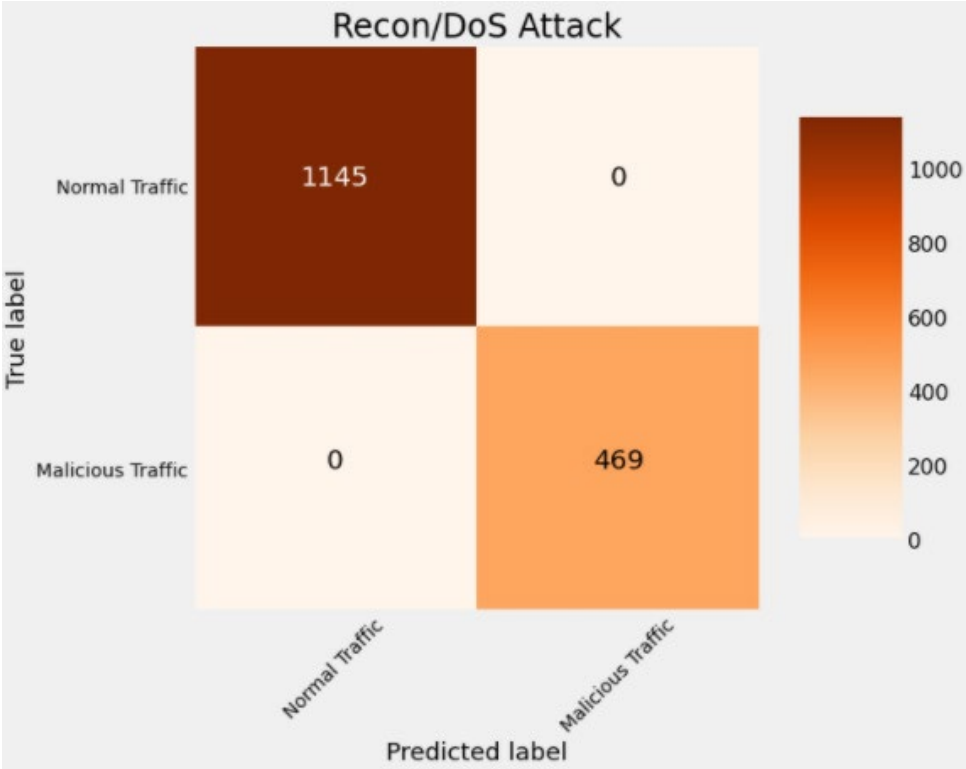


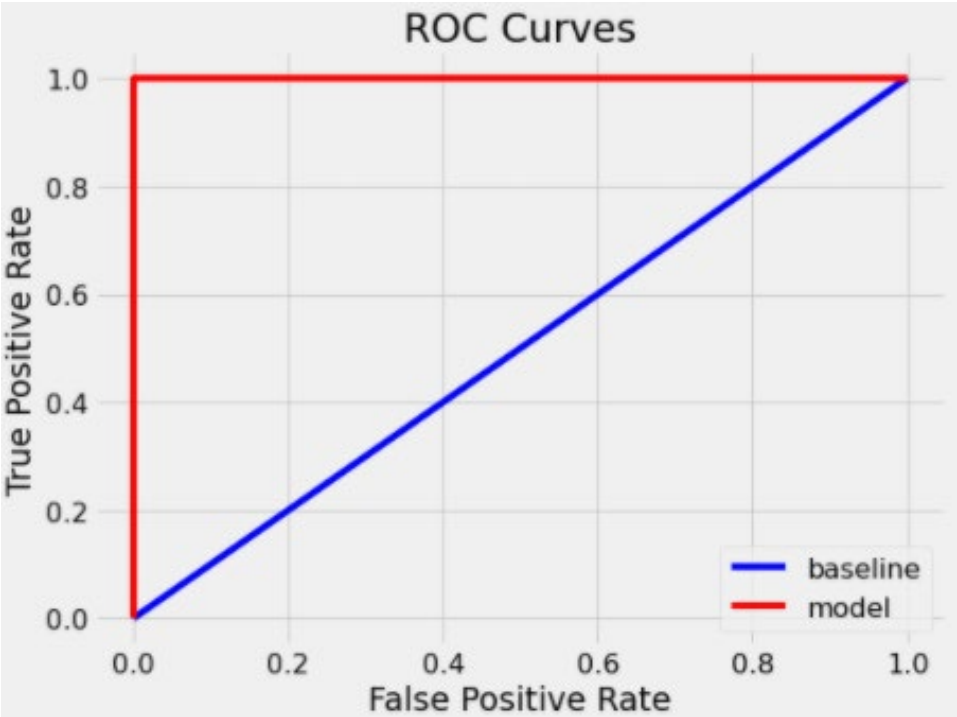
Google Home Device:



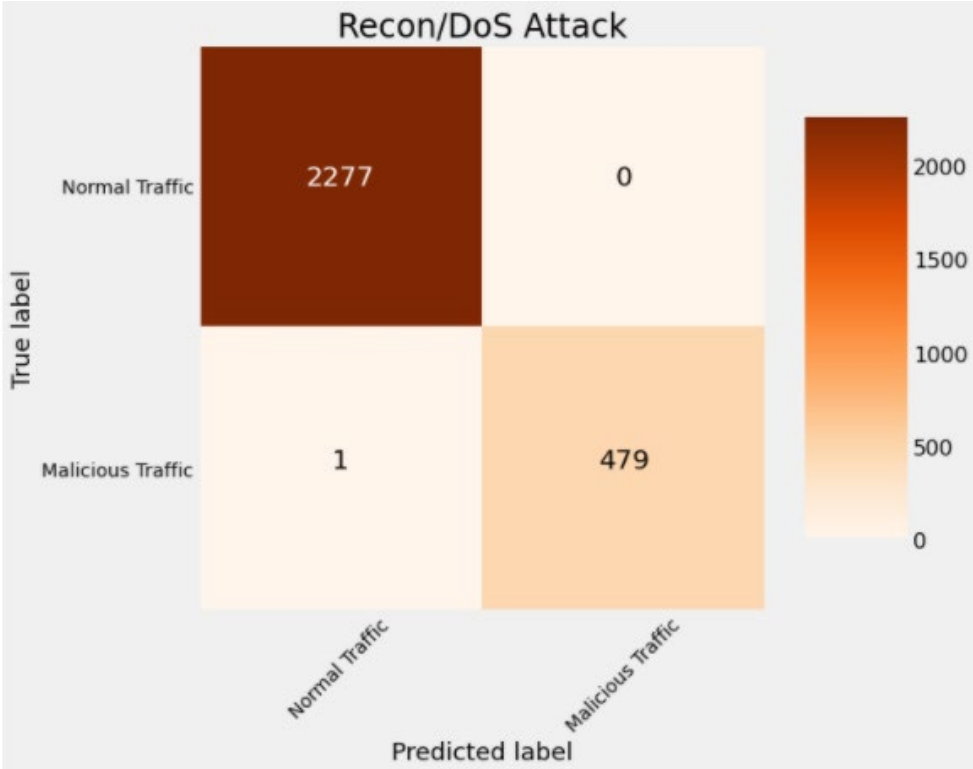


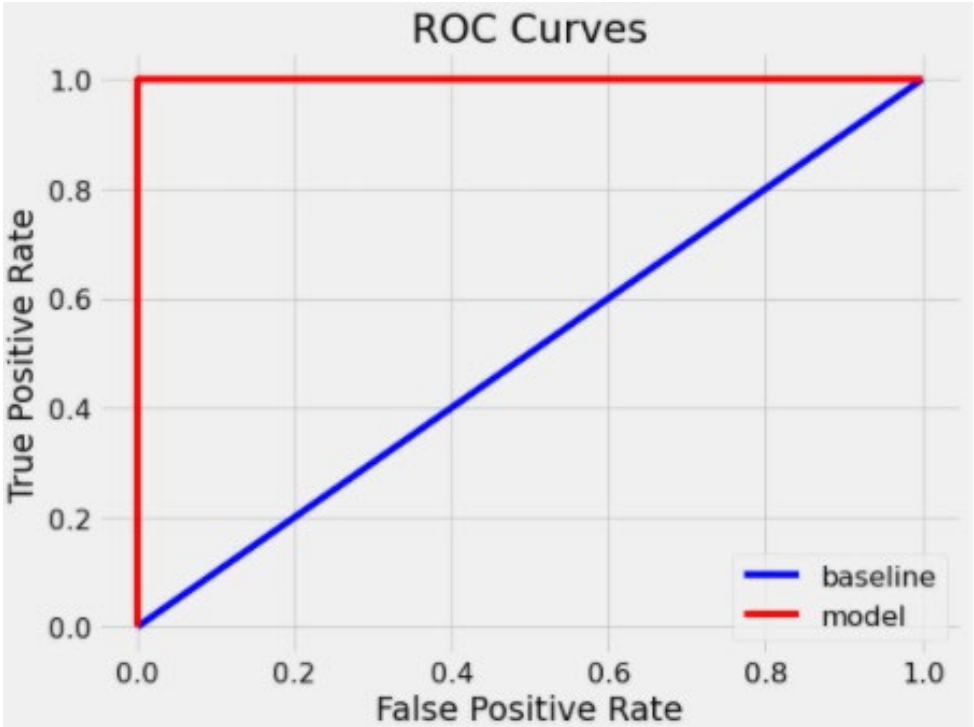
Roomba Smart Vacuum:





Amazon Firestick:



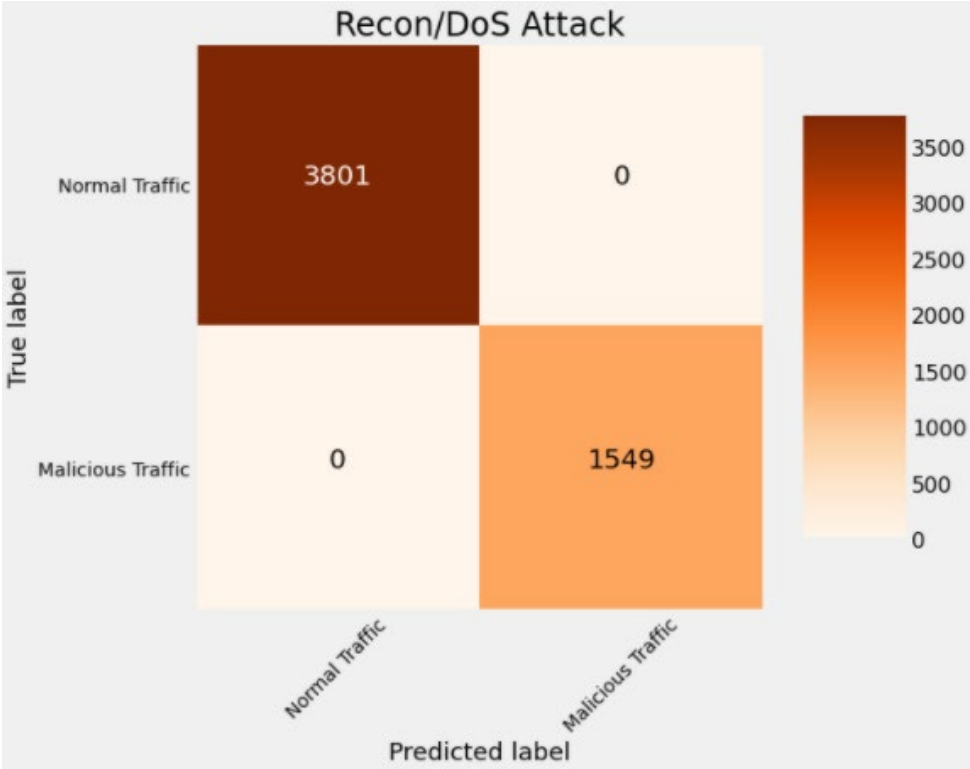


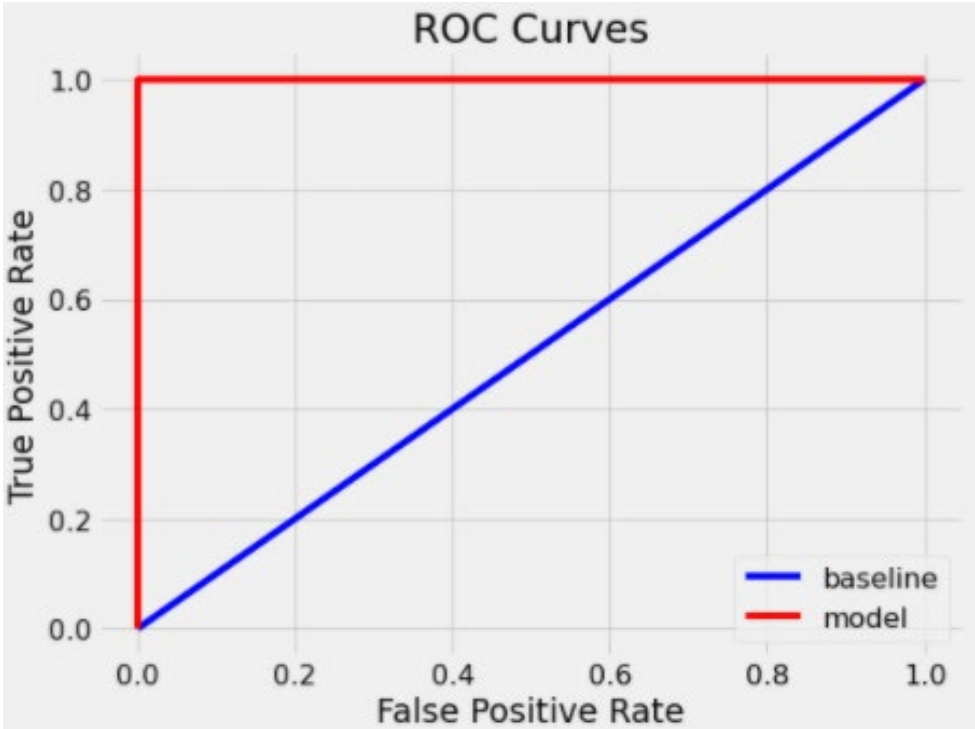
5.1.2 Random Forest Metrics

Device	Accuracy	Recall	Precision	TP	FP	TN	FN
Smart Bulb	99.9%	.999	1	3801	0	1548	1
Google Home	99.9%	.999	.999	6650	2	1437	1
Roomba	100%	.1	1	1145	0	469	0
Firestick	99.9%	.999	1	2277	0	479	1

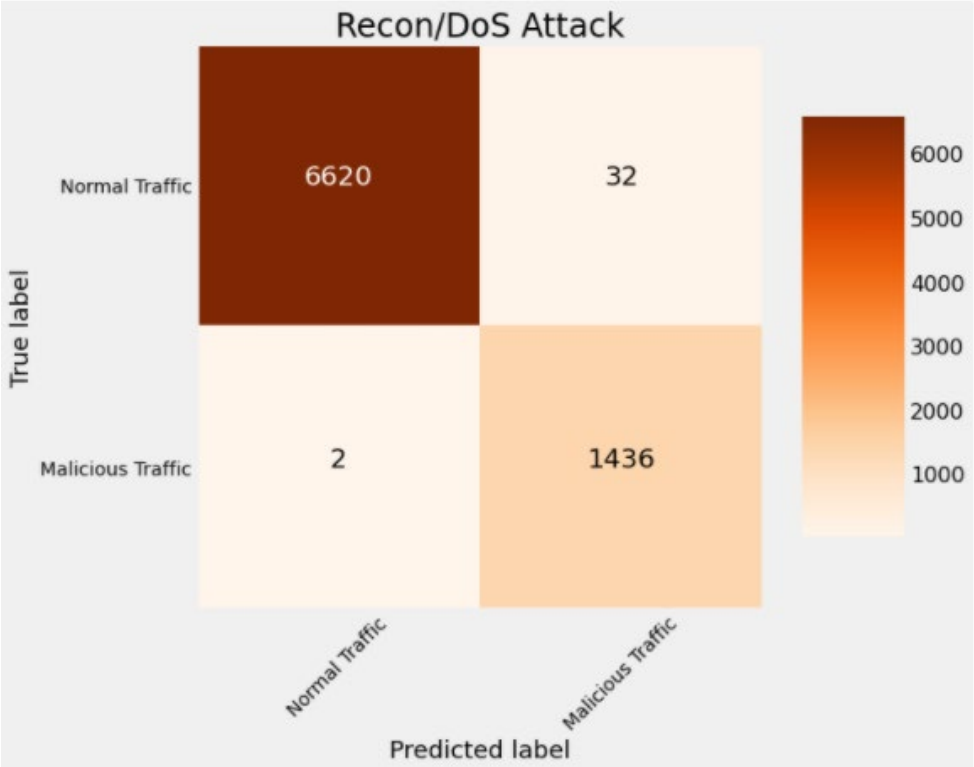
5.1.3 Support Vector Machine Confusion Matrices and ROC Curve

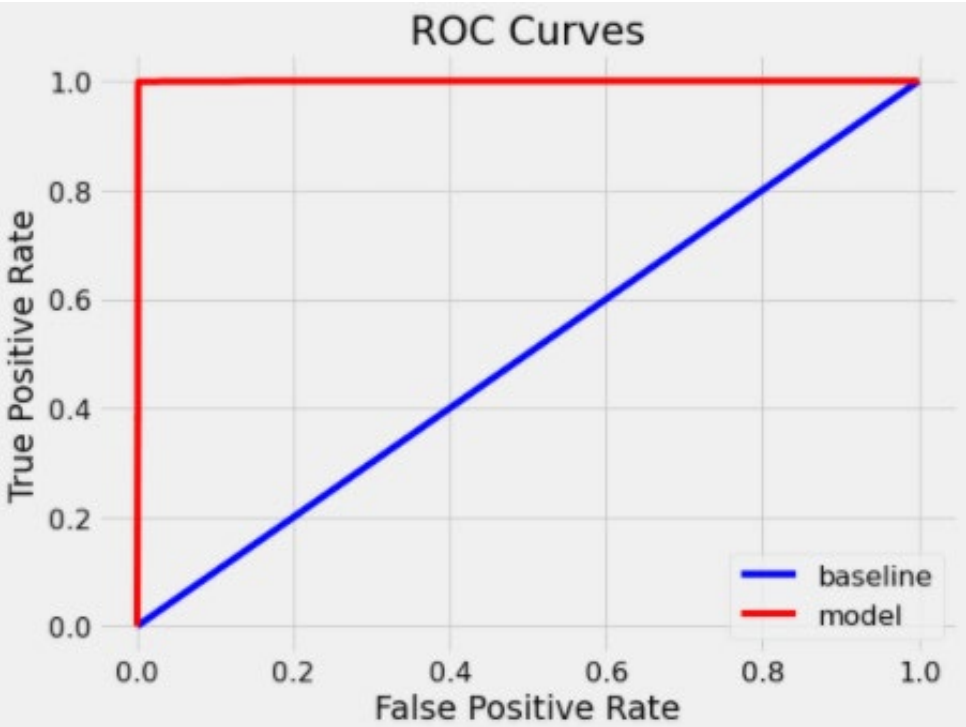
Smart Bulb Device:



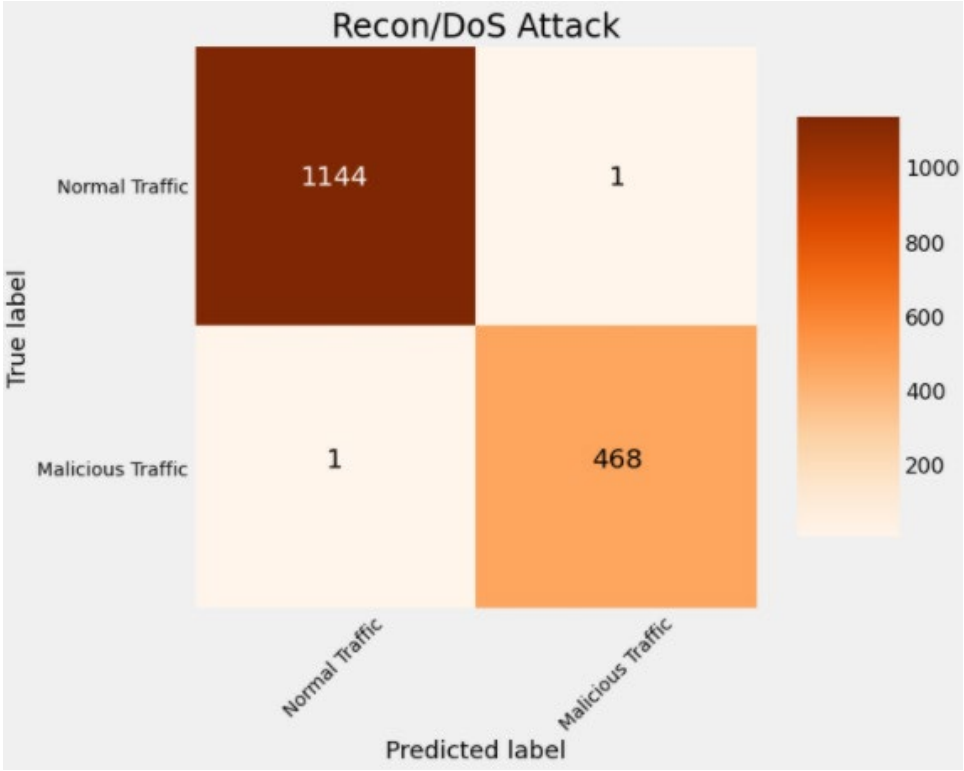


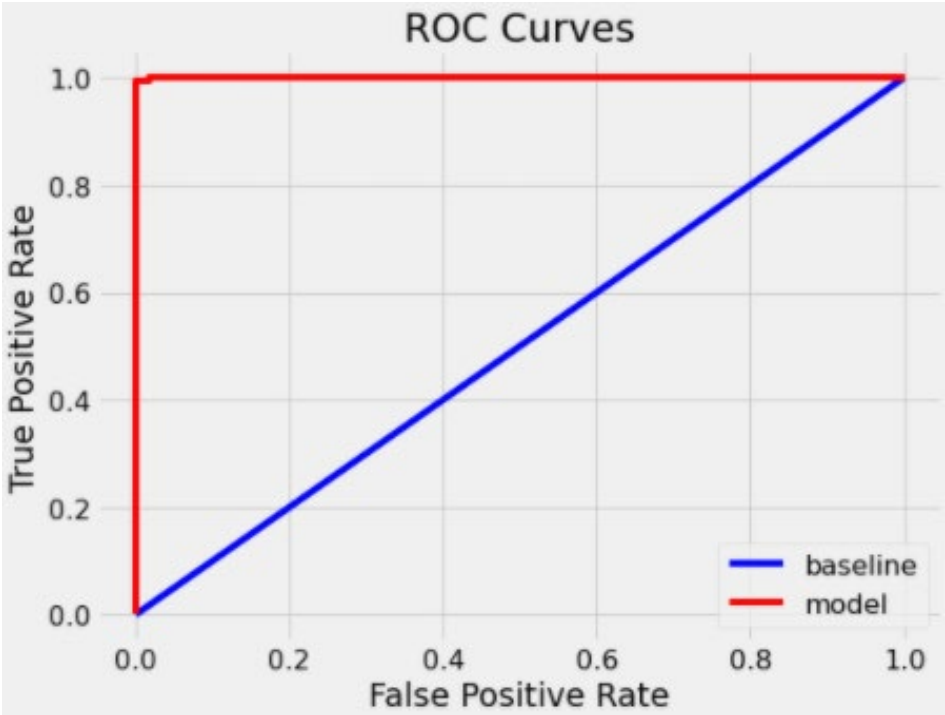
Google Home Device:



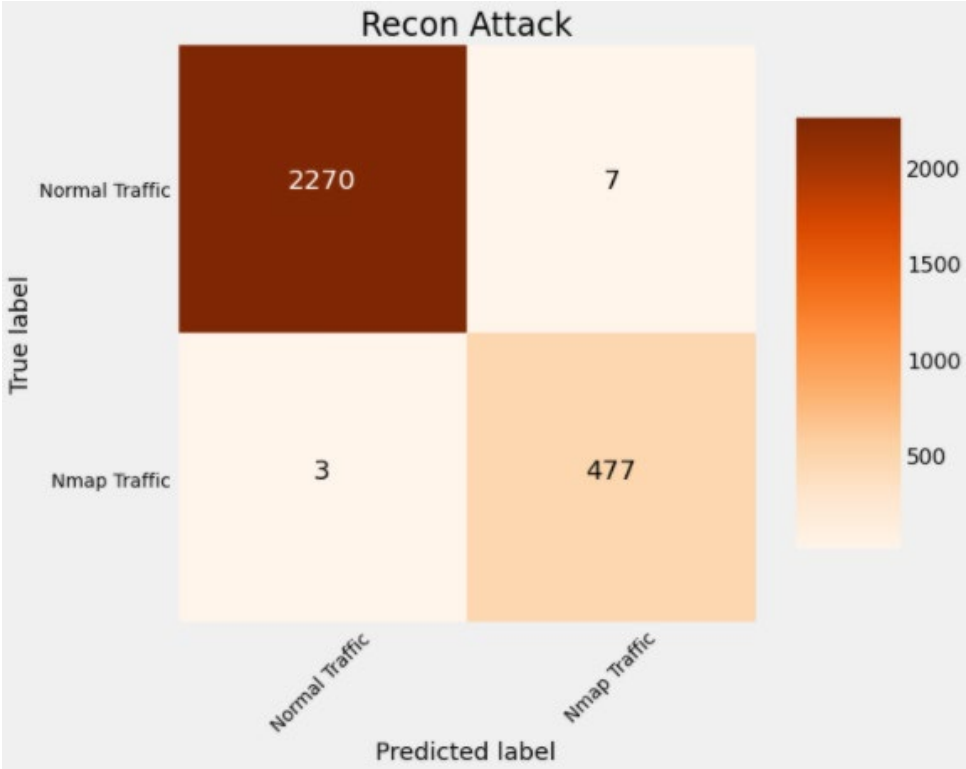


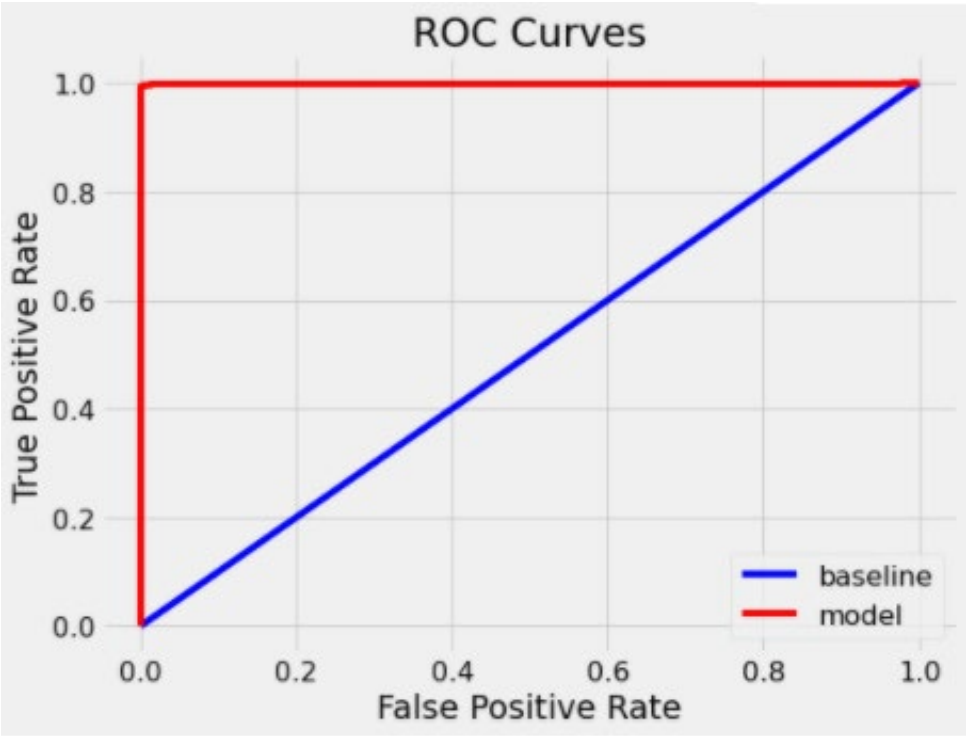
Roomba Smart Vacuum:





Amazon Firestick:



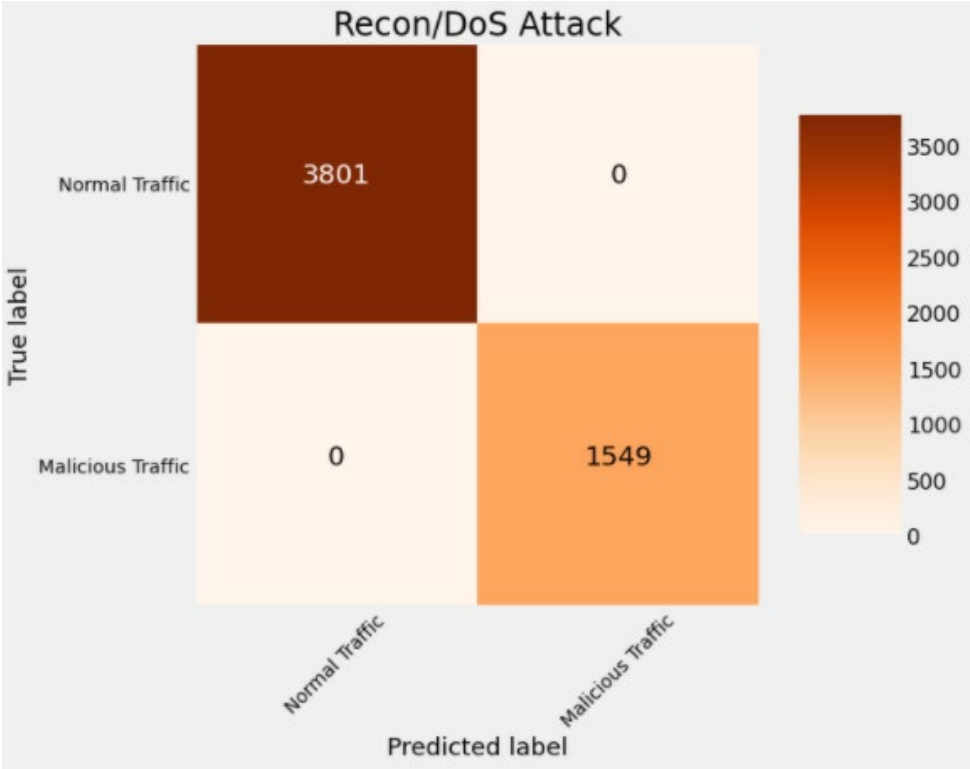


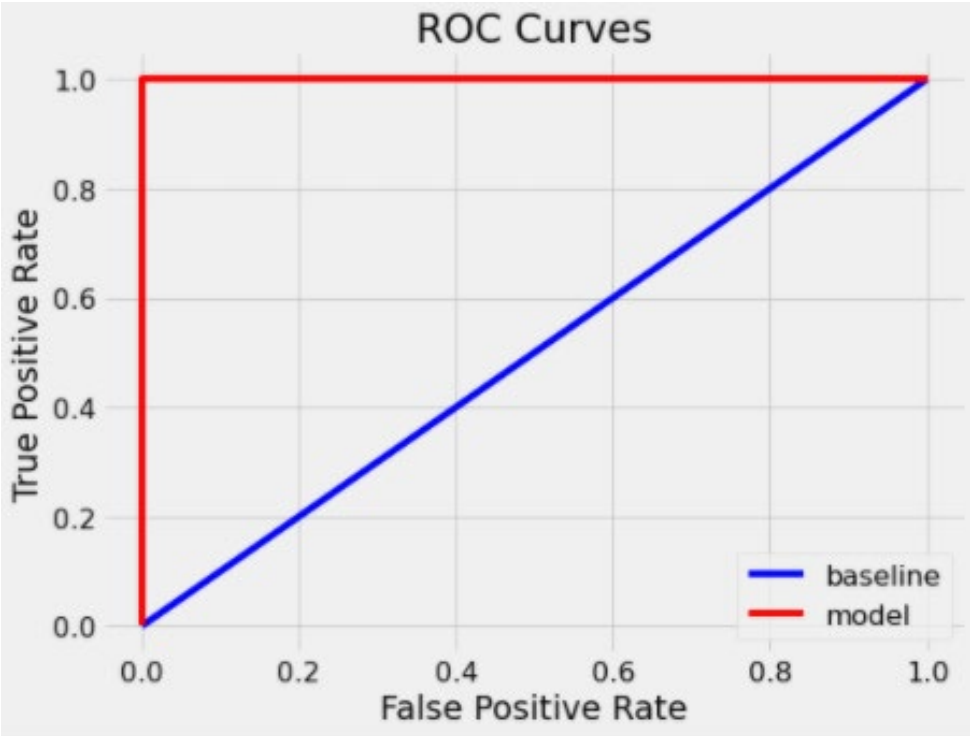
5.1.4 Support Vector Machine Metrics

Device	Accuracy	Recall	Precision	TP	FP	TN	FN
Smart Bulb	100%	1.0	1.0	3801	0	1549	0
Google Home	99.9%	.996	.999	6620	32	1436	2
Roomba	99.9%	.999	.999	1144	1	468	1
Firestick	99.9%	.999	.997	2270	7	477	3

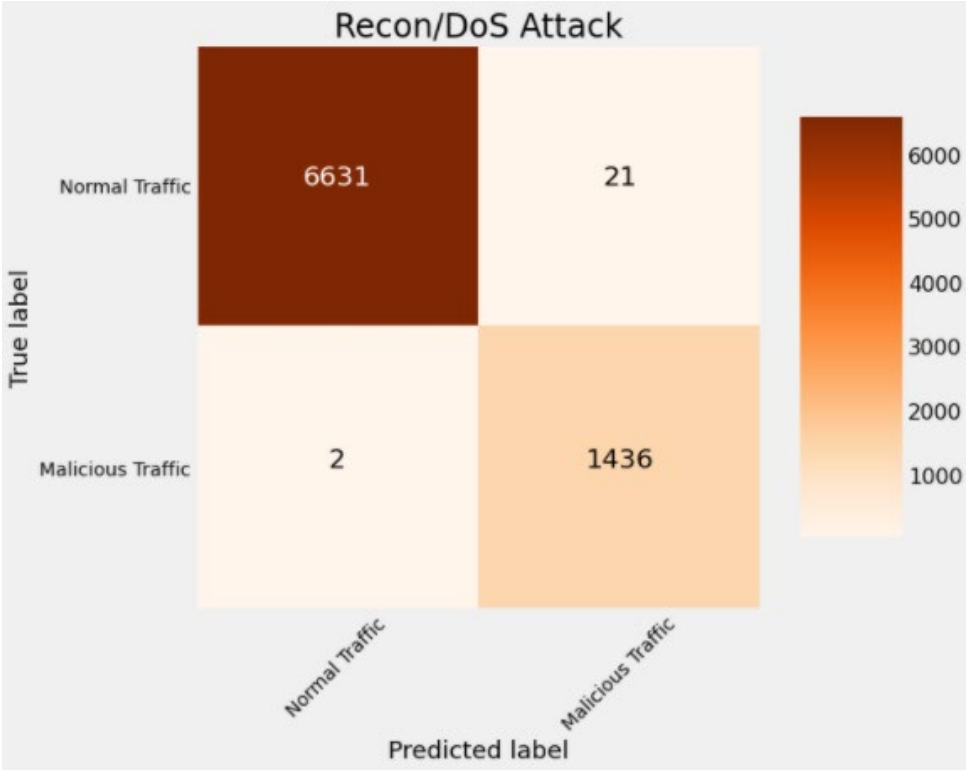
5.1.5 Multi-Layer Perceptron Confusion Matrices and ROC Curves

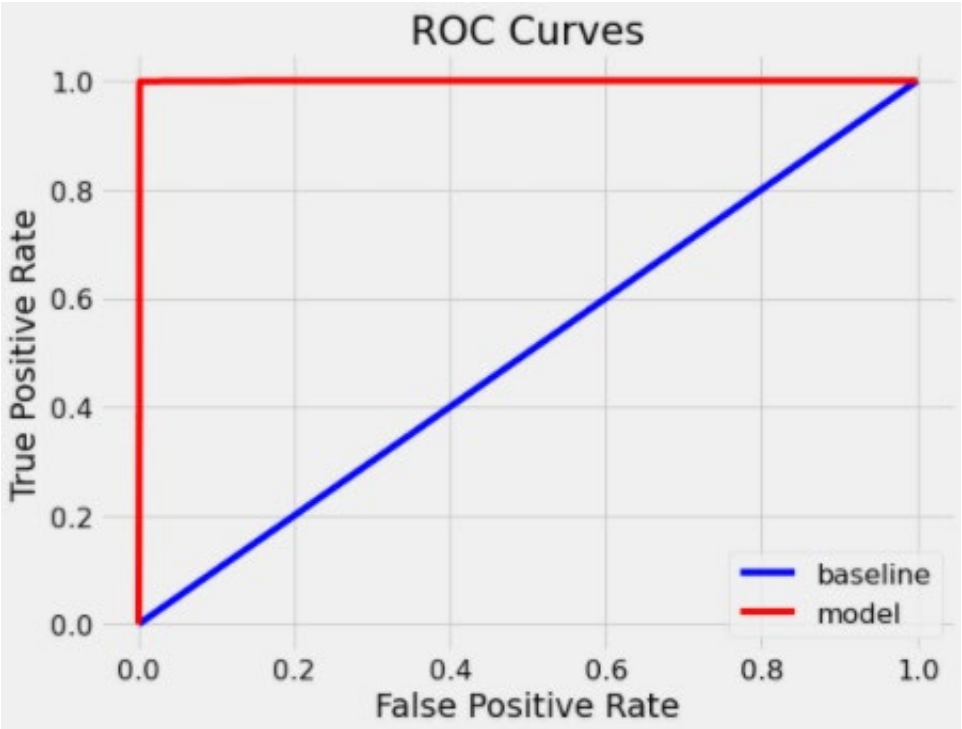
Smart Bulb Device:



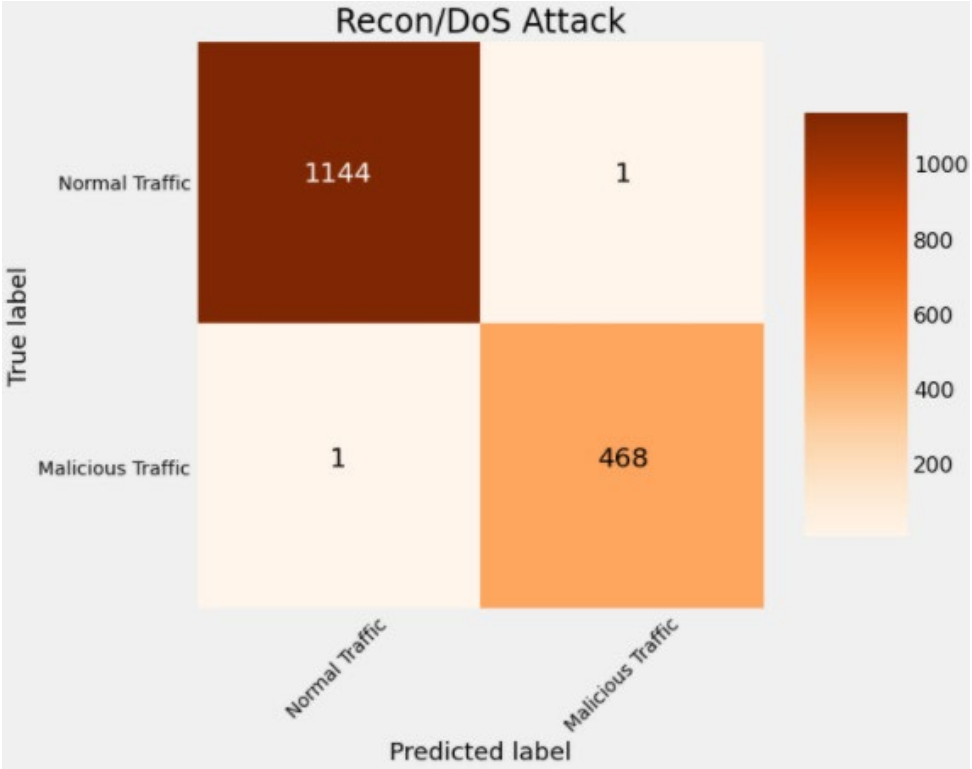


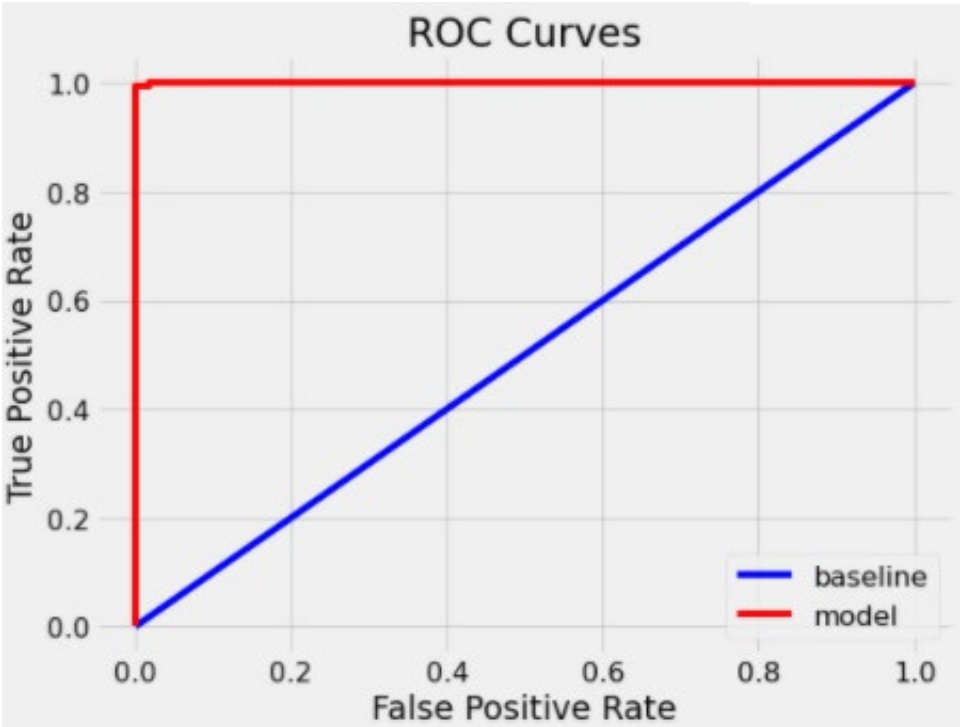
Google Home Device:



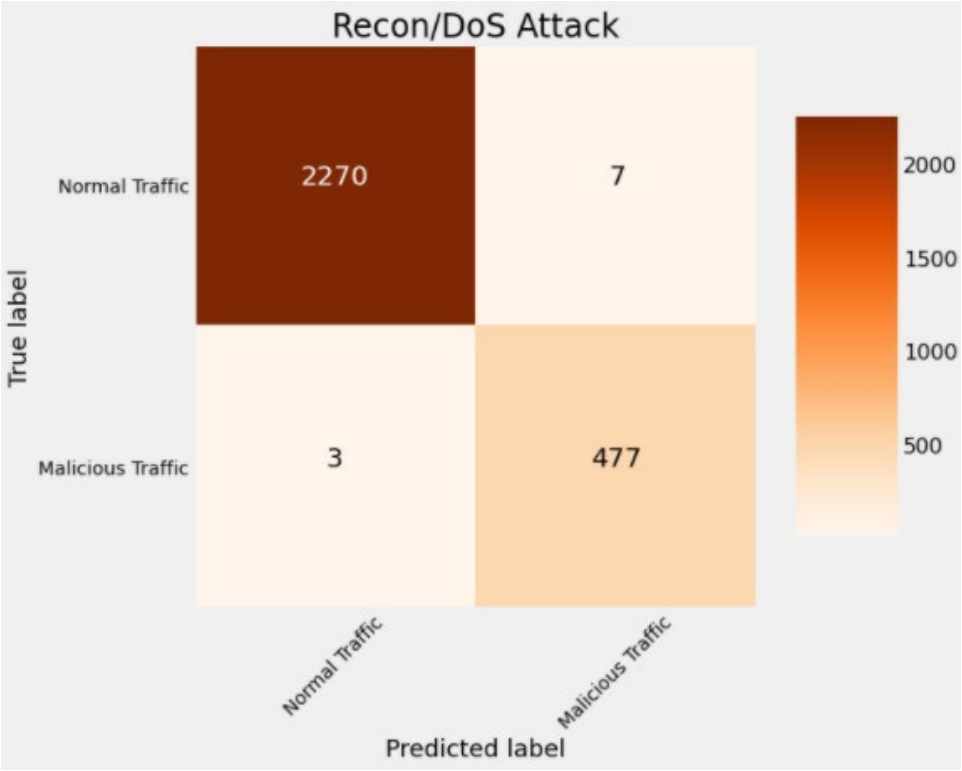


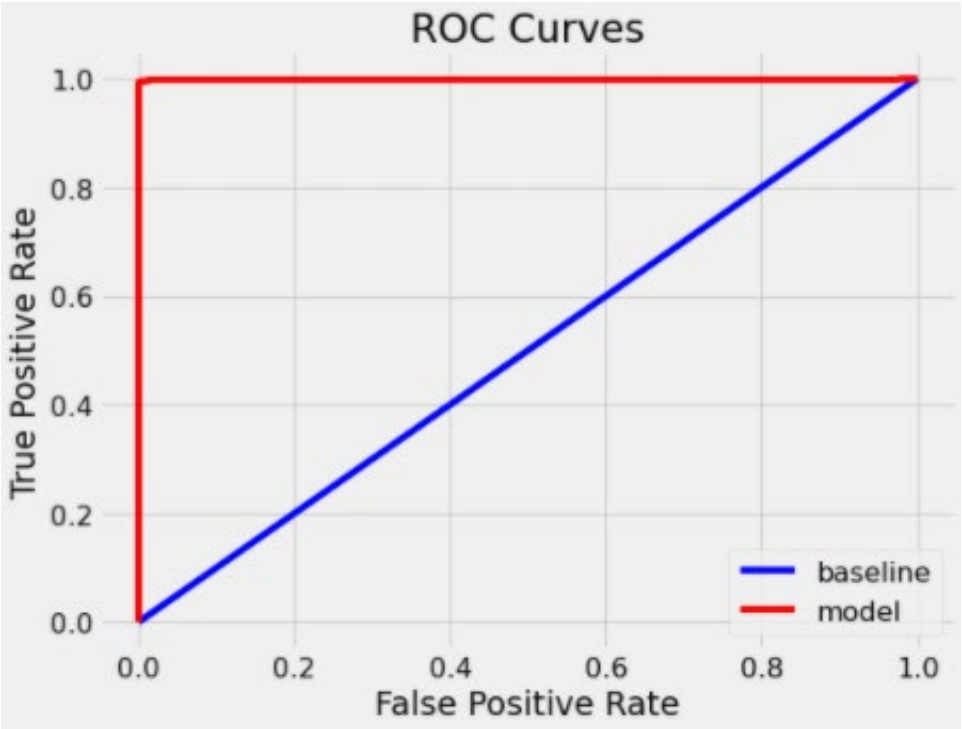
Roomba Smart Vacuum:





Amazon Firestick:





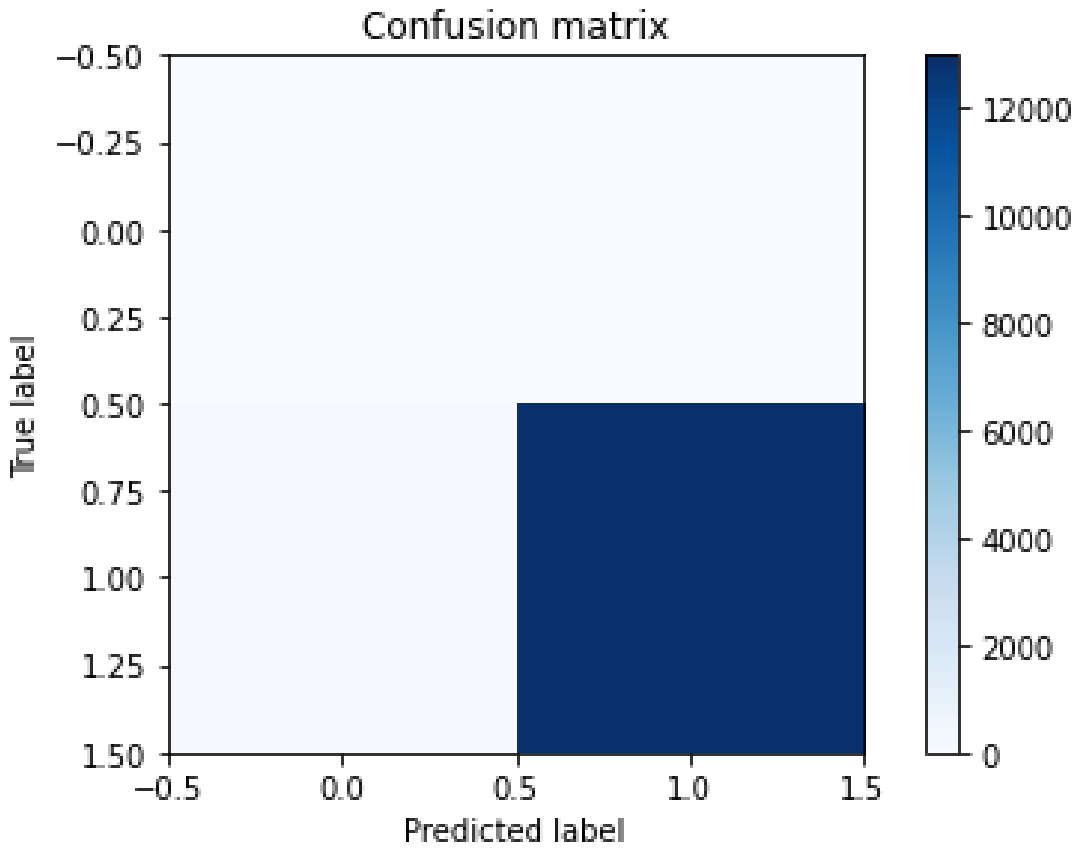
5.1.6 Multi-Layer Perceptron Metrics

Device	Accuracy	Recall	Precision	TP	FP	TN	FN
Smart Bulb	100%	1.0	1.0	3801	0	1549	0
Google Home	99.7%	.997	.999	6631	21	1436	2
Roomba	99.9%	.999	.999	1144	1	468	1
Firestick	99.6%	.997	.997	2270	7	477	3

5.2 Unsupervised Learning

5.2.1 Evaluating the Autoencoder

The performance of an autoencoder algorithm can be depicted via a confusion matrix, an example of which is shown in Figure 17. Since the autoencoder is trained to only identify normal traffic, we can assume that all values should be identified as normal traffic.



```
**********************
0.9440000000000007
TP:12973
FP:0
TN:0
FN:127
Accuracy:0.9903053435114504
Precision:1.0
Recall:0.9903053435114504
F1:0.9951290607141487
```

Figure 17. Confusion matrix displaying results of trained autoencoder.

In addition, the autoencoder uses the Mean Squared Error (MSE) function as the loss function to measure the difference or error between the reconstructed and the original input. It can be formally defined by the formula in Figure 18 [25].

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=0}^{N}(\vec{x_i} - \hat{\vec{x_i}})^2$$

Figure 18. Formula for the mean squared error (MSE).

All we need to do is to find a loss threshold that distinguishes the normal from fraudulent data. If the MSE loss value for an input feature is above the loss threshold, then the input feature is an anomaly. Otherwise, if the MSE loss value for an input feature is below the threshold then the input feature is normal. Figure 19 shows the resulting anomalies after capturing live network traffic. Although the autoencoder

technique was not used to actively distinguish network traffic due to poor results, this is an area which can benefit from further research and a proposed improvement for future work.

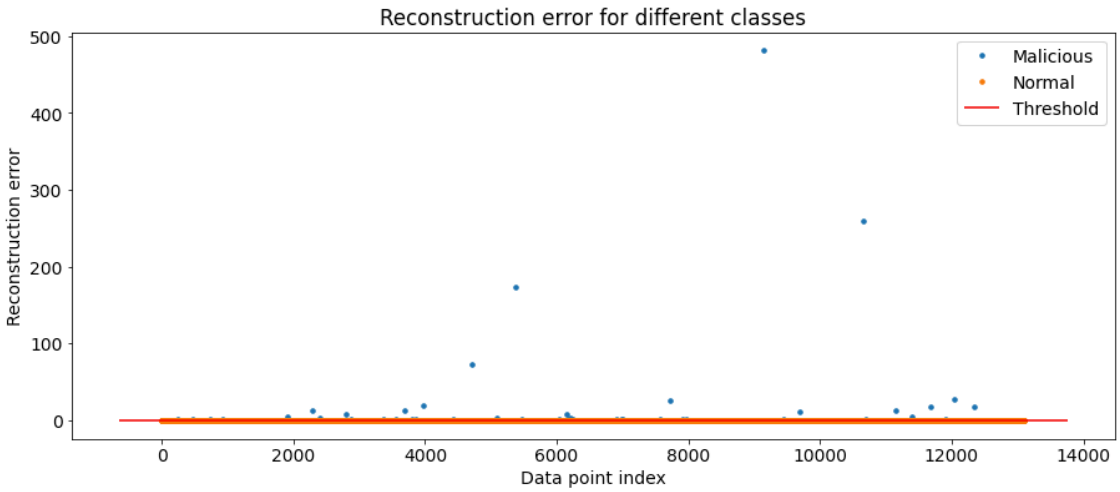


Figure 19. Analysis of network traffic after calculating the mean squared error.

6. CONCLUSION AND FUTURE WORK

In this work, we developed a methodology for detecting malicious network activity in a Smart Home environment using machine learning techniques deployed on a Raspberry Pi 4. By leveraging Zeek to generate logs as input, it was found that trained models can detect malicious activity with high accuracy based on the data extracted. In addition, we were able to learn which features were significant enough to identify common IoT attacks such as denial-of-service and reconnaissance gathering. Given these results, we can conclude that real-time intrusion detection can be performed in a smart home environment, using a low-cost solution, in a limited resource environment with a Raspberry Pi. Training and deploying this solution in a Smart Home environment has the added benefit of being fully customizable and adaptable to unique environments. We also found that basic machine learning algorithms such as Random Forest and Support Vector Machines can be effectively used when classifying network traffic. Although unsupervised learning techniques may also help improve the solution, further research is needed to develop and integrate an effective solution into the data pipeline. This is one of several areas which can be used to pursue as a future direction for this work. Some other areas include collecting data and training models for additional attacks, self-learning by introducing a feedback loop, and performing intrusion prevention.

7. REFERENCES

[1] "2020 Global Threat Report: CrowdStrike." 2020. *Crowdstrike.com*. September 29. https://www.crowdstrike.com/resources/reports/2020-crowdstrike-global-threat-report/.

[2] Abadi, Martín, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg Corrado, et al. 2015. "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems." *TensorFlow*. Google Research. https://www.tensorflow.org/.

[3] Estevez-Tapiador, Juan M, Pedro Garcia-Teodoro, and Jesus E Diaz-Verdejo. 2004. "Anomaly Detection Methods in Wired Networks: a Survey and Taxonomy." *Computer Communications* 27 (16): 1569–84. doi:https://doi.org/10.1016/j.comcom.2004.07.002.

[4] Ferrag, Mohamed Amine, Leandros Maglaras, Sotiris Moschoyiannis, and Helge Janicke. 2020. "Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study." *Science Direct* 50: 102419. doi:https://doi.org/10.1016/j.jisa.2019.102419.

[5] García, S., M. Grill, J. Stiborek, and A. Zunino. 2014. "An Empirical Comparison of Botnet Detection Methods." *Computers & Security* 45: 100–123. doi:10.1016/j.cose.2014.05.011.

[6] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2017. "Ch. 14 Autoencoders." Essay. In *Deep Learning*, 500–523. Cambridge, MA: The MIT Press.

[7] Hettich, S., and S. D. Bay. 1999. "Welcome to the UCI Knowledge Discovery in Databases Archive." *UCI KDD Archive*. University of California, Department of Information and Computer Science. http://kdd.ics.uci.edu/.

[8] Kasongo, Sydney Mambwe, and Yanxia Sun. 2020. "A Deep Learning Method with Wrapper Based Feature Extraction for Wireless Intrusion Detection System."

Computers & Security 92: 101752. doi:10.1016/j.cose.2020.101752.

[9] Khamis, Rana Abou, M. Omair Shafiq, and Ashraf Matrawy. 2020. "Investigating Resistance of Deep Learning-Based IDS against Adversaries Using Min-Max Optimization." *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*. doi:10.1109/icc40277.2020.9149117.

[10] Kherlenchimeg, Zolzaya, and Naoshi Nakaya. 2020. "A Deep Learning Approach Based on Sparse Autoencoder with Long Short-Term Memory for Network Intrusion Detection." *IEEJ Transactions on Electronics, Information and Systems* 140 (6): 592–99. doi:10.1541/ieejeiss.140.592.

[11] Kim, Kwangjo. 2020. "Intrusion Detection System Using Deep Learning and Its Application to Wi-Fi Network." *IEICE Transactions on Information and Systems* E103.D (7): 1433–47. doi:10.1587/transinf.2019ici0001.

[12] Kluyver, Thomas, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, et al. 2016. "Jupyter Notebooks—a Publishing Format for Reproducible Computational Workflows." *The IOS Press*, 87–90. doi:10.3233/978-1-61499-649-1-87.

[13] Kramer, Mark A. 1991. "Nonlinear Principal Component Analysis Using Autoassociative Neural Networks." *AIChE Journal* 37 (2): 233–43. doi:10.1002/aic.690370209.

[14] Li, Yanmiao, Yingying Xu, Zhi Liu, Haixia Hou, Yushuo Zheng, Yang Xin, Yuefeng Zhao, and Lizhen Cui. 2020. "Robust Detection for Network Intrusion of Industrial IoT Based on Multi-CNN Fusion." *Measurement* 154: 107450. doi:10.1016/j.measurement.2019.107450.

[15] Magán-Carrión, Roberto, Daniel Urda, Ignacio Díaz-Cano, and Bernabé Dorronsoro. 2020. "Towards a Reliable Comparison and Evaluation of Network Intrusion Detection Systems Based on Machine Learning Approaches." *Applied Sciences* 10 (5): 1775. doi:10.3390/app10051775.

[16] Manimaran, A., D. Chandramohan, S. G. Shrinivas, and N. Arulkumar. 2020. "A Comprehensive Novel Model for Network Speech Anomaly Detection System Using Deep Learning Approach." *International Journal of Speech Technology* 23 (2): 305–13. doi:10.1007/s10772-020-09693-z.

[17] Marsland, Stephen. 2015. *Machine Learning: an Algorithmic Perspective*. Boca Raton, FL: CRC Press.

[18] McCulloch, W. S., and W. Pitts. 1943. "A Logical Calculus of the Ideas Immanent in Nervous Activity." *Bulletin of Mathematical Biophysics* 5: 115–33. doi:https://doi.org/10.1007/BF02478259.

[19] Mirsky, Yisroel, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection." *Proceedings 2018 Network and Distributed System Security Symposium*. doi:10.14722/ndss.2018.23204.

[20] Moustafa, Nour, and Jill Slay. 2016. "The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set." *Information Security Journal: A Global Perspective* 25 (1-3): 18–31. doi:10.1080/19393555.2015.1125974.

[21] Muhammad,, Ghulam, Sahil Garg, and M. Shamim Hossain. 2020. "Stacked Autoencoder-Based Intrusion Detection System to Combat Financial Fraudulent." *IEEE Internet of Things Journal*, 1–1. doi:10.1109/JIOT.2020.3041184.

[22] Murphey, Kevin P. 2012. "Ch. 1 Introduction; Ch 28. Deep Learning." Essay. In *Machine Learning: A Probabilistic Perspective*, 1–1005. Cambridge, MA: MIT Press.

[23] Nguyen, Quoc Phong, Kar Wai Lim, Dinil Mon Divakaran, Kian Hsiang Low, and Mun Choon Chan. 2019. "GEE: A Gradient-Based Explainable Variational Autoencoder for Network Anomaly Detection." *2019 IEEE Conference on Communications and Network Security (CNS)*. doi:10.1109/cns.2019.8802833.

[24] Pedregosa, Fabian, View Profile, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, et al. 2011. "Scikit-Learn: Machine Learning in Python." *The Journal of Machine Learning Research*. November 1. https://dl.acm.org/doi/10.5555/1953048.2078195.

[25] Samarabandu, Jagath, and Sunanda Gamage. 2020. "Deep Learning Methods in Network Intrusion Detection: A Survey and an Objective Comparison." *Journal of Network and Computer Applications. Academic Press*. August 5. https://www.sciencedirect.com/science/article/pii/S1084804520302411.

[26] Sharafaldin, Iman, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. doi:10.5220/0006639801080116.

[27] Shekhawat, Anish Singh, Fabio Di Troia, and Mark Stamp. 2019. "Feature Analysis of Encrypted Malicious Traffic." *Expert Systems with Applications* 125: 130–41. doi:10.1016/j.eswa.2019.01.064.

[28] Shone, Nathan, Tran Nguyen Ngoc, Vu Dinh Phai, and Qi Shi. 2018. "A Deep Learning Approach to Network Intrusion Detection." *IEEE Transactions on Emerging Topics in Computational Intelligence* 2 (1): 41–50. doi:10.1109/TETCI.2017.2772792.

[29] Sovilj, Dušan, Paul Budnarain, Scott Sanner, Geoff Salmon, and Mohan Rao. 2020. "A Comparative Evaluation of Unsupervised Deep Architectures for Intrusion Detection in Sequential Data Streams." *Expert Systems with Applications* 159: 113577. doi:10.1016/j.eswa.2020.113577.

[30] Stratosphere IPS. 2020. "Aposemat IoT-23: A Labeled Dataset With Malicious And Benign IoT Network Traffic." *Stratosphere IPS*. Stratosphere IPS. January 22. https://www.stratosphereips.org/blog/2020/1/22/aposemat-iot-23-a-labeled-dataset-with-malicious-and-benign-iot-network-traffic.

[31] Summerville, Douglas H., Kenneth M. Zach, and Yu Chen. 2015. "Ultra-

Lightweight Deep Packet Anomaly Detection for Internet of Things Devices." *2015 IEEE 34th International Performance Computing and Communications Conference (IPCCC)*. doi:10.1109/pccc.2015.7410342.

[32] SuperCowPowers. 2021. "SuperCowPowers/Zat." *GitHub*. Accessed January 18. https://github.com/SuperCowPowers/zat/blob/master/examples/zeek_to_pandas.py.

[33] Thing, Vrizlynn L. L. 2017. "IEEE 802.11 Network Anomaly Detection and Attack Classification: A Deep Learning Approach." *2017 IEEE Wireless Communications and Networking Conference (WCNC)*. doi:10.1109/wcnc.2017.7925567.

[34] Zarpelão, Bruno Bogaz, Rodrigo Sanches Miani, Cláudio Toshio Kawakani, and Sean Carlisto de Alvarenga. 2017. "A Survey of Intrusion Detection in Internet of Things." *Journal of Network and Computer Applications* 84 (1084-8045): 25–37. doi:https://doi.org/10.1016/j.jnca.2017.02.009.

[35] Zavrak, Sultan, and Murat Iskefiyeli. 2020. "Anomaly-Based Intrusion Detection From Network Flow Features Using VAE." *IEEE Access* 8 (108346). doi:10.1109/ACCESS.2020.3001350.

[36] "The Zeek Network Security Monitor." 2017. *Zeek: An Open Source Network Security Monitoring Tool*. https://zeek.org/